

Politechnika
Śląska

Politechnika Śląska

Wydział Automatyki Elektroniki I Informatyki

Kierunek Elektronika i Telekomunikacja

Praca dyplomowa magisterska

Implementacja oscyloskopu cyfrowego w układzie SoC

Autor: inż. Dawid Broda

Kierujący pracą: dr hab. Robert Czerwiński, prof. Pol. Śl.

Gliwice, marzec 2021

Wykaz ważniejszych oznaczeń

- ADC - ang. Analog-to-Digital Converter
- ASIC - ang. Application-specific integrated circuit
- AXI - ang. Advanced eXtensible Interface
- DSP - ang. Digital Signal Processor
- FPGA - ang. Field Programmable Gate Array
- GPIO - ang. General-Purpose Input/Output
- GSPS - ang. Gigasamples per second
- ILA - ang. Integrated Logic Analyzer
- MSPS - ang. Megasamples per second
- PL - Programmable Logic
- PS - ang. Processing System
- SoC - ang. System-on-a-Chip
- VNC - ang. Virtual Network Computing

Wykaz ważniejszych oznaczeń **1**

Rozdział I **4**

Wprowadzenie 4

1.1 Wstęp 4

1.2 Cel i zakres pracy 5

Rozdział II **7**

Analiza możliwych rozwiązań 7

2.1 Przegląd dostępnych na rynku urządzeń pomiarowych 7

2.2 Przegląd dostępnych płytek rozwojowych i architektur jednostek centralnych 9

2.2.1 Komputery jednopłytkowe 9

2.2.2 Płytki z układem mikrokontrolera 10

2.2.3 Płytki z układem FPGA 12

2.2.4 Płytki z układem SoC Xilinx Zynq 14

2.2.5 Podsumowanie 16

Rozdział III **20**

Projekt części sprzętowej oscyloskopu 20

3.1 Analiza interfejsów dostępnych w układzie SoC 20

3.1.1 Interfejs AXI4 20

3.1.2 Interfejs AXI4-Lite 24

3.2 Implementacja bloków podstawowych 24

3.3 Implementacja bloków pomiarowych 28

3.3.1 Układ z interfejsem równoległym 28

3.3.2 Układ z interfejsem AXI 34

3.4 Przygotowanie plików projektu Vivado 36

Rozdział IV **38**

Projekt części programowej 38

4.1 Środowisko PYNQ 38

4.2 Przygotowanie środowiska PYNQ 39

4.3 Konfiguracja systemu Linux oraz instalacja wymaganych pakietów 41

4.3.1 Pierwsze uruchomienie układów 41

4.3.2 Konfiguracja dostępu do internetu 42

4.3.3 Instalacja oraz konfiguracja pakietów 43

4.4 Aplikacja GUI 45

4.4.1 Interfejs aplikacji 45
4.4.2 Integracja aplikacji z bibliotekami PYNQ 47

Rozdział V **49**
Badania opracowanego oscyloskopu 49
5.1 Test aplikacji GUI 49
5.2 Badanie wydajności opracowanego systemu 63

Rozdział VI **69**
Podsumowanie i wnioski 69

Literatura **71**

Rozdział I

Wprowadzenie

1.1 Wstęp

Postęp technologiczny i rozwój układów mikroprocesorowych oraz układów logicznych FPGA sprawił, że technologie SoC (ang. System-on-a-Chip) integrujące oba układy stały się powszechnie dostępne i posiadają szeroką gamę zastosowań w otaczającym nas świecie. Obecne układy SoC z mikroprocesorem ARM oferują moce obliczeniowe na tyle wysokie, że możliwe jest zainstalowanie pełnego i rozbudowanego systemu operacyjnego opartego o jądro Linux. Natomiast część FPGA układu oprócz samej logiki posiada różnego rodzaju układy wejścia/wyjścia, układy DSP, jak również bloki pamięci. Układy SoC z logiką FPGA rozbudowują funkcjonalność procesora, rozszerzając dostępne peryferia, pozwalają na przyspieszenie części obliczeń w logice oraz umożliwiają zaimplementowanie własnych układów.

Funkcjonalność układów FPGA sprawia, że idealnie nadają się do zaawansowanego przetwarzania złożonych sygnałów oraz komunikacji z szybkimi interfejsami i coraz częściej wykorzystywane są przez producentów przenośnych urządzeń pomiarowych jako jednostki centralne, zastępując powszechnie stosowane mikrokontrolery w przystawkach oscyloskopowych. Tradycyjne urządzenia wymagają komputerów PC do sterowania układem oraz odpowiedniego programu do wyświetlania uzyskanych danych. Do poprawnego działania wymagane są również szybki interfejs komunikacyjny oraz sterowniki wspierające wiele platform i systemów operacyjnych. W tym przypadku zastosowanie układów SoC takich jak Zynq-7000 lub Zynq UltraScale+ firmy Xilinx czy SoC Cyclone V firmy Intel znacznie upraszczają implementację układu do obsługi pomiarów, umożliwiając sterowanie i wyświetlanie za pomocą

standardowych protokołów oraz narzędzi. Dodatkowo platforma Zynq UltraScale+ umożliwia pracę bez konieczności podłączania komputera PC.

1.2 Cel i zakres pracy

Celem pracy jest implementacja części cyfrowej oscyloskopu w układzie SoC. Cel pracy obejmuje między innymi stworzenie platformy dla układów Zynq-7000 oraz Zynq UltraScale+, zaimplementowanie różnych sposobów przekazywania danych i porównanie ich wydajności, przygotowanie systemów dla płytki Ultra96-v2 producenta 96boards oraz PYNQ-Z2 firmy TUL, wraz ze środowiskiem PYNQ. Wymagane jest skonfigurowanie systemu Linux do zdalnego zarządzania układem, przygotowanie odpowiednich funkcji dla przetwarzania próbek sygnału, stworzenie aplikacji okienkowej demonstrującej przetwarzany sygnał oraz przeprowadzenie testów dla obu platform. Następnie przeprowadzone zostanie badanie wydajności interfejsu AXI opracowanych platform dla obu układów z wykorzystaniem środowiska PYNQ firmy Xilinx.

Szczegółowy zakres pracy obejmuje:

- Przegląd dostępnych na rynku układów ADC oraz metod przesyłu danych pomiędzy układami o wysokiej częstotliwości próbkowania.
- Przygotowanie architektur dla struktury logiki z wykorzystaniem oprogramowania Vivado.
- Zaprojektowanie modułu do obsługi przetwornika z interfejsem równoległym.
- Integracja stworzonych modułów ze środowiskiem PYNQ.
- Konfiguracja systemu Linux oraz instalacja wymaganych pakietów do zarządzania zdalnego układem.
- Stworzenie aplikacji okienkowej w środowisku Python z wykorzystaniem dostępnych bibliotek do wyświetlania i przetwarzania zmierzonego sygnału.

- Przeprowadzenie badań obejmujących pomiary wydajności interfejsu AXI, porównanie rodzin układów ZYNQ oraz przedstawienie otrzymanych wyników.

Rozdział II

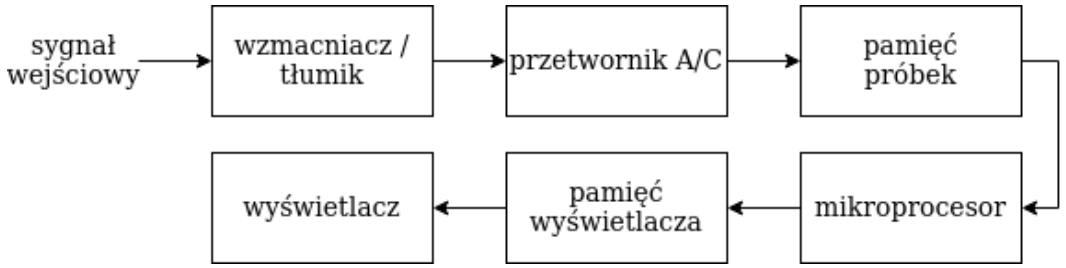
Analiza możliwych rozwiązań

2.1 Przegląd dostępnych na rynku urządzeń pomiarowych

Dokonywanie pomiarów oraz testów jest bardzo ważnym procesem podczas tworzenia i rozwoju wszelkiego rodzaju układów i urządzeń. Obecnie postęp technologiczny w dziedzinie cyfrowego przetwarzania sygnałów, przyczynił się do rozpowszechnienia cyfrowych układów pomiarowych, zastępując tradycyjne oscyloskopy analogowe ich cyfrowymi odpowiednikami oraz przenośnymi przystawkami, umożliwiające wyświetlanie oraz wstępne przetwarzanie pobranych danych. Dostępne na rynku cyfrowe układy pomiarowe można podzielić ze względu na sposób, w jaki przetwarzane są obserwowane sygnały, wyróżniając cztery typy [1]:

- oscyloskopy cyfrowe DSO (ang. Digital Storage Oscilloscopes),
- oscyloskopy cyfrowe DPO (ang. Digital Phosphor Oscilloscopes),
- oscyloskopy cyfrowe MSO (ang. Mixed Signal Oscilloscopes),
- oscyloskopy cyfrowe MDO (ang. Mixed Domain Oscilloscopes).

Podstawowymi elementami większości oscyloskopów cyfrowych, wymieniając od wejścia sygnału do wyjścia, są: wzmacniacz lub tłumik sygnału, przetwornik analogowo-cyfrowy, pamięć próbek, mikroprocesor, pamięć wyświetlacza oraz wyświetlacz. Droższe urządzenia posiadają również rozbudowane układy wyzwalania, łączność bezprzewodową, zaawansowane funkcje matematyczne lub mogą wykorzystywać technologię fosforu cyfrowego.



Rysunek 1: Uproszczony schemat blokowy oscyloskopu cyfrowego

Przystawki oscyloskopowe w celu wyświetlenia pomiarów, wymagają podłączenia do komputera PC oraz uruchomienia programu sterującego i wizualizującego dane, dostarczonego przez producenta. Parametry przystawek oraz ich funkcjonalność różnią się w zależności od ceny. W najtańszych układach dominuje typ DSO, który cechuje się zastosowaniem układów cyfrowych z niską mocą obliczeniową i niewielką pamięcią próbek. W takich urządzeniach układ centralny najczęściej oparty jest na mikrokontrolerze i odpowiada tylko za zbieranie oraz przekazanie próbek do podłączonego komputera, gdzie następuje dalsze przetwarzanie i wyświetlanie mierzonego sygnału. Natomiast za konwersję sygnału analogowego odpowiedzialne są układy ADC zintegrowane wewnątrz mikrokontrolera lub dedykowane przetworniki najczęściej odpowiadające za pomiar na kilku kanałach, dołączone poprzez magistrale szeregowe I2C i SPI. Koszt takiego urządzenia zawiera się w przedziale od 100 zł do 500 zł i najczęściej pozwala na dwukanałowy pomiar sygnałów o paśmie do 20 MHz i próbkowaniu do 1 MSPS.

Urządzenia typu DSO powyżej tej ceny oferują wyższą rozdzielczość, wyższe częstotliwości pasma i próbkowania. Dostępne są również urządzenia typu MSO, rozszerzające funkcjonalność pomiaru sygnałów analogowych dodatkowo o generatory funkcyjne oraz analizator stanów logicznych i protokołów sygnałowych. Jednostki centralne zastosowane w takich urządzeniach to najczęściej układy DSP, wydajne mikrokontrolery, układy FPGA lub układy ASIC, pozwalające na wstępne lub całkowite przetwarzanie sygnałów, zanim zostaną przesłane do komputera. Za konwersję sygnału odpowiadają układy ADC

dedykowane na każdy kanał, o rozdzielczościach osiągających do 16 bitów i próbkowaniu do 10 GSPS. Ze względu na wysokie częstotliwości próbkowania przetworników, do przesyłu danych wykorzystywane są magistrale równoległe oraz interfejsy JESD204B i JESD204C. Koszt takich układów może wynosić od 1200 zł do 60000 zł i zależy od wydajności jednostki centralnej, przetworników oraz zastosowanych elementów w torze analogowym. Najdroższe układy oferują możliwość pomiarów częstotliwości radiowych oraz łączy światłowodowych.

2.2 Przegląd dostępnych płytek rozwojowych i architektur jednostek centralnych

Jednym z elementów pracy było zapoznanie się z architekturą jednostek centralnych dostępnych na rynku, a następnie wybranie odpowiedniej platformy dla projektu. W tym celu należało się przyjrzeć dostępnym płytkom rozwojowym i deweloperskim, które tworzone są w celu przybliżenia działania oraz możliwości danej architektury. Płytki wyposażone są w układy potrzebne do poprawnego działania urządzenia oraz dodatkowe peryferia dostępne do wykorzystania przez użytkownika, co znacznie ułatwia prototypowanie własnych projektów. Producenci najczęściej udostępniają również podstawowe oprogramowanie, systemy operacyjne oraz przykładowe projekty dla danych układów, które pozwalają na uruchomienie płytki z praktycznymi aplikacjami oraz tworzenie nowych na własne potrzeby. W płytkach do zastosowań komercyjnych najczęściej spotykanymi jednostkami centralnymi są mikrokontrolery oraz mikroprocesory, natomiast układy SoC, FPGA i DSP można znaleźć częściej u samych producentów układów cyfrowych. Zarówno same układy, jak i formy płytek występują w wielu wariantach, oferujące różne częstotliwości pracy, napięcia logiki lub dostępne peryferia. Mnogość wariantów układów oraz płytek rozwojowych pozwala na selekcję konkretnej platformy, eliminując konieczność tworzenia prototypów w pierwszych etapach projektu, zmniejszając zarówno koszty, jak i czas rozwoju.

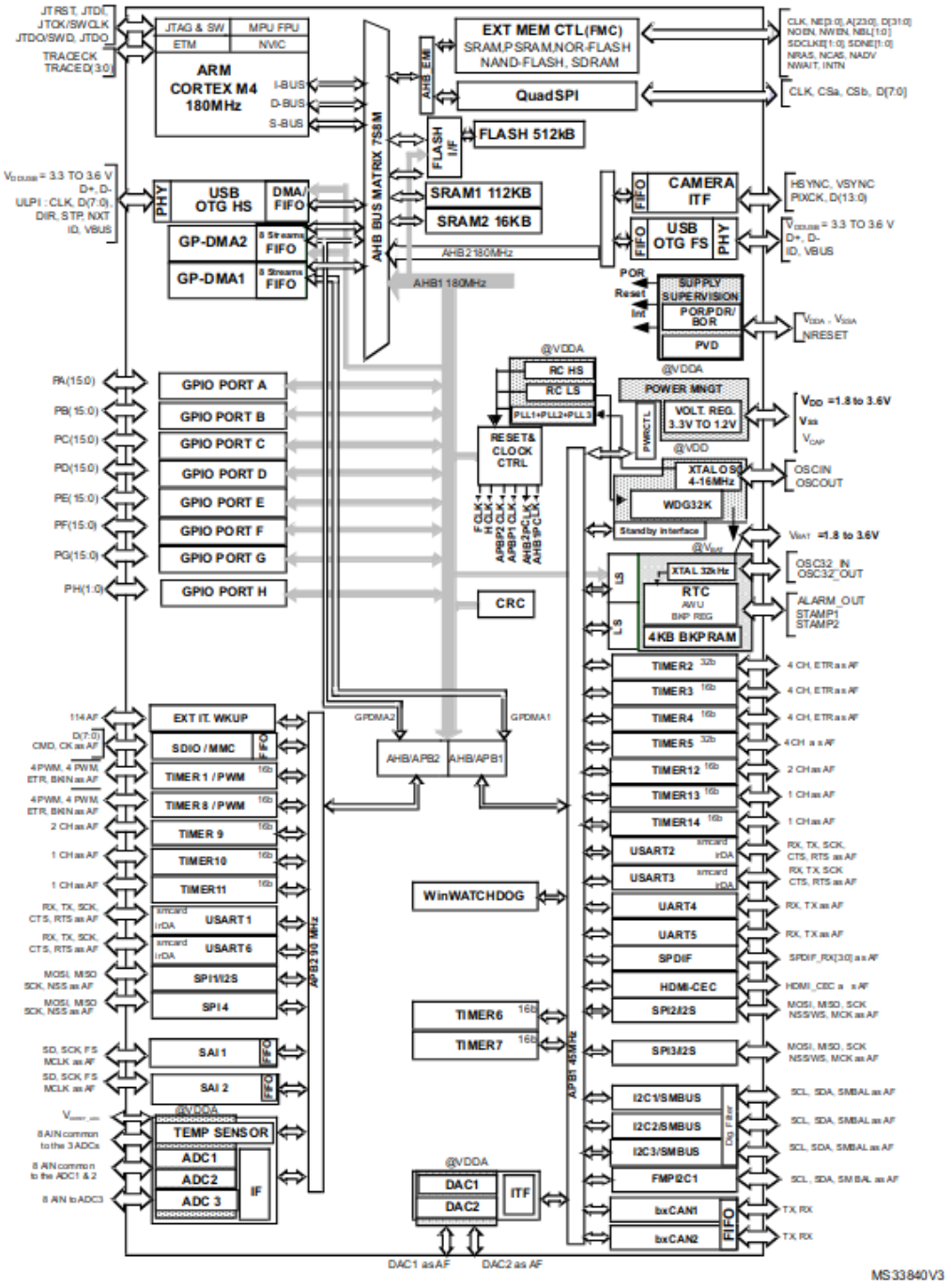
2.2.1 Komputery jednopłytkowe

Komputery jednopłytkowe w skrócie SBC (ang. Single Board Computer) są urządzeniami z architekturą zbliżoną do komputerów osobistych i najczęściej wpisują się w kategorię płytek rozwojowych. Komputer jednopłytkowy wyposażony jest w jednostkę centralną, którą zazwyczaj jest mikroprocesor lub układ SoC, oprócz tego posiadają pamięć RAM, pamięć stałą lub port na pamięć Flash oraz porty GPIO, USB, audio, wideo i Ethernet. Nowsze urządzenia obecnie oferują również łączność bezprzewodową. W przeciwieństwie do tradycyjnych komputerów z modularnym systemem wszystkie elementy są zintegrowane na jednej płytce PCB. Dodatkowo są to urządzenia charakteryzujące się niewielkimi rozmiarami, obsługą popularnych magistral komunikacyjnych oraz wysoką energooszczędnością kosztem możliwości obliczeniowych. Dzięki dostępności wyprowadzeń GPIO możliwe jest w prosty sposób rozbudowanie płytek o dodatkowe peryferia, często wykorzystujące magistrale SPI lub I2C. Najczęściej spotykane są urządzenia z jednostką centralną opartą na architekturze ARM Cortex-A, pozwalającą na uruchomienie pełnego systemu operacyjnego z jądrem Linux. Jedną z wielu zalet tego rodzaju urządzeń są możliwości graficzne, pozwalające na wyprowadzenie sygnału wideo przy pomocy zintegrowanych w systemie układów, jak i zawartych w jądrze sterowników oraz oprogramowania. Pozwala to na podłączenie do urządzenia monitora przy pomocy standardowych wyjść wideo. Nie jest wymagana od użytkownika implementacja obsługi wyświetlacza oraz oprogramowanie do generacji sygnałów wideo, co znacznie upraszcza złożoność projektu oraz prezentację danych przy pomocy ogólnodostępnych paneli. Do najpopularniejszych standardów, na których oparte są projekty płytek SBC, należą między innymi: ITX, Raspberry Pi model B oraz standardy 96Boards i BeagleBone, które umożliwiają tworzenie uniwersalnych układów peryferyjnych dla wielu urządzeń, niezależnie od producenta lub jednostki centralnej.

2.2.2 Płytki z układem mikrokontrolera

Mikrokontroler jest układem scalonym charakteryzujący się zintegrowanymi układami takimi jak: jednostka centralna, pamięć RAM oraz układy czasowe i peryferyjne. Obecnie coraz częściej mikrokontrolery posiadają zintegrowaną pamięć programu, kontrolery transmisji szeregowej, układy kontrolujące watchdog, układy ADC/DAC oraz proste czujniki. Niska cena i szeroki zakres możliwości sprawiają, że mikrokontrolery stają się bardzo atrakcyjne dla zastosowań, niewymagających wysokich mocy obliczeniowych, dostępnych w tradycyjnych procesorach. Obecnie mikrokontrolery możemy znaleźć we wszystkich nowych urządzeniach elektronicznych, jak również w urządzeniach codziennego użytku, od sterowników w nowoczesnych żarówkach, przez sprzęt AGD, aż po urządzenia obsługi inteligentnych budynków.

Na rynku istnieje wiele architektur mikrokontrolerów, obecnie dominującą jest ARM Cortex-M, natomiast pozostałe najczęściej związane są z konkretnym producentem czipu, na przykład STM8 firmy ST lub AVR i PIC firmy Microchip. Seria Cortex-M jest architekturą zarówno sprawdzoną, jak i bardzo dobrze udokumentowaną, oferuje ona 32-bitową jednostkę centralną mogącą osiągnąć częstotliwość pracy do 600 MHz. Ponadto wyposażona jest w magistralę AMBA (ang. Advanced Microcontroller Bus Architecture), która pozwala na komunikację jednostki centralnej z układami peryferyjnymi oraz pamięcią. Wymienione funkcjonalności pozwalają na potencjalne wykorzystanie układu jako przystawki podłączonej do komputera PC, służącej do pomiaru sygnałów o niskiej częstotliwości. Natomiast wykorzystanie mikrokontrolera jako autonomicznego urządzenia jest utrudnione ze względu na ograniczone zasoby pamięci potrzebnej do przechowywania próbek oraz mocy obliczeniowej, potrzebnej do przetwarzania zebranych próbek. Przykładowy schemat mikrokontrolera opartego na rdzeniu Cortex-M w układzie STM32F446x [2] przedstawiono na Rysunku 2.



Rysunek 2: Schemat układu STM32F446x [2].

2.2.3 Płytki z układem FPGA

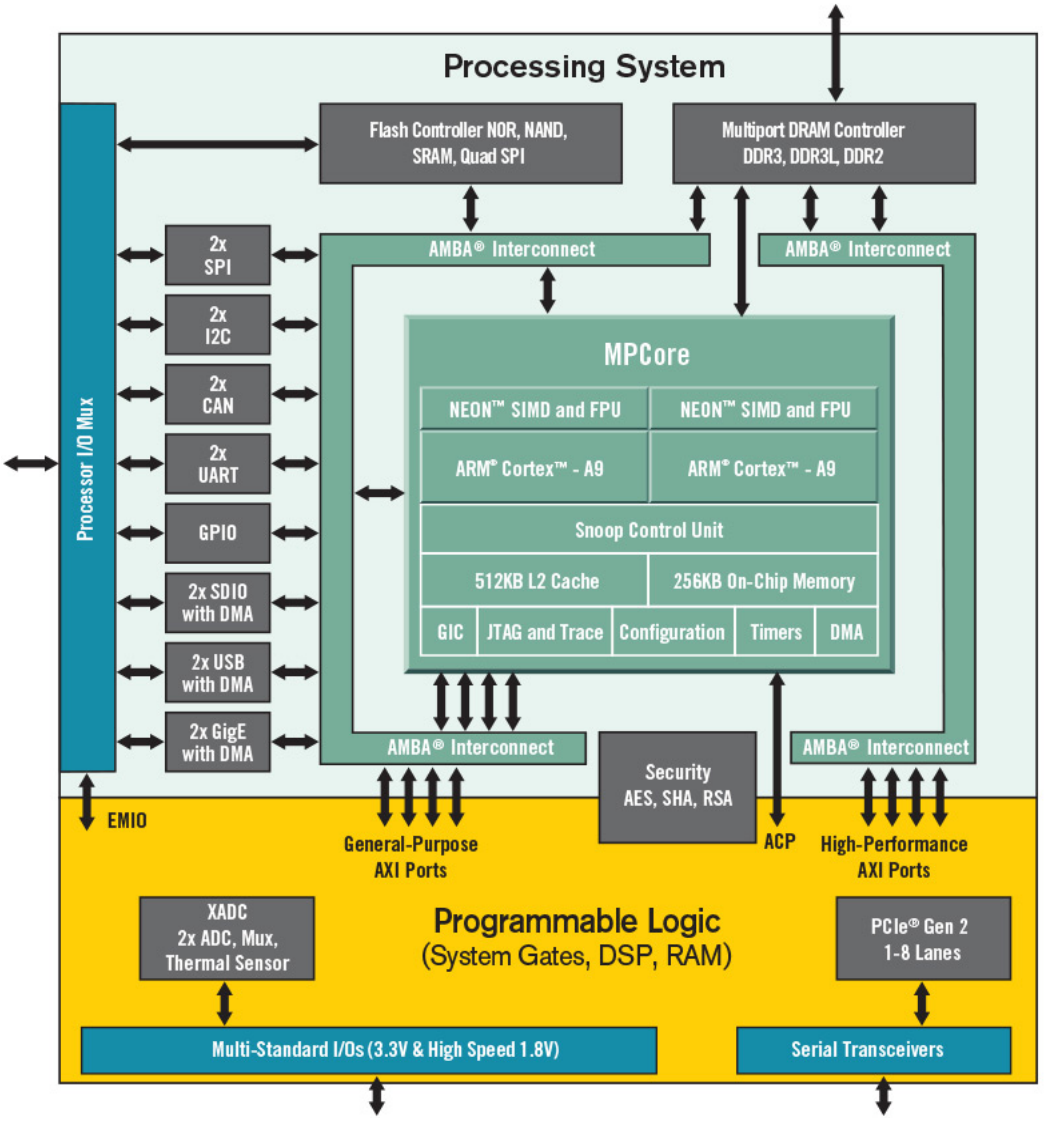
Układ logiki programowalnej jest układem o bardzo wysokim stopniu scalenia zbudowanym z podstawowych elementów, takich jak: bloki LUT (ang. Look-up table), które odpowiadają za realizację funkcji logicznych, przerzutniki typu D,

bloki pamięci, multipleksery, programowalne połączenia oraz banki wyprowadzeń sygnałów. Najnowsze układy mogą zawierać również: pamięć RAM, mnożarki, bloki DSP, wbudowane procesory, wyprowadzenia szybkich sygnałów oraz kontrolery szybkich magistral takich jak PCIe, czy Ethernet. Układ FPGA między innymi pozwala użytkownikowi na zdefiniowanie własnej architektury procesora i układów ASIC, stworzenie własnego peryferium, zaimplementowanie bloków do akceleracji obliczeń, wykorzystanie w przyśpieszeniu uczenia maszynowego oraz rozwoju sztucznej inteligencji. Układy FPGA pozwalają również rozszerzyć projektowaną architekturę o mikroprocesor typu softcore, jak na przykład Microblaze firmy Xilinx, który w całości jest zaimplementowany w logice i pozwala na wykonywanie skompilowanego kodu lub uruchomienie pełnego systemu operacyjnego. Dodatkowo projektowanie i implementacja własnych układów wspomagana jest poprzez wykorzystywanie gotowych bloków IP oraz oprogramowania dostarczone przez producenta, takie jak na przykład Vivado, Vitis, Vitis HLS lub Petalinux firmy Xilinx. Podstawowymi funkcjami programu Vivado są: tworzenie modułów za pomocą tradycyjnych języków opisu sprzętu, generacja bloków IP, synteza kodu, implementacji w danym układzie FPGA, programowanie układów, debugowanie przy pomocy modułu ILA analizatora stanów logicznych zaimplementowanego w logice oraz przeprowadzanie symulacji syntezowanego kodu. Program Vitis odpowiada za tworzenie aplikacji typu „bare-metal” i generację obrazów systemu dla procesorów Zynq oraz Microblaze. Dodatkowo pozwala na akcelerację wielu algorytmów oraz przetwarzania danych za pomocą udostępnionych przez producenta bibliotek. Vitis HLS jest narzędziem, które odpowiada za syntezę układów napisanych przy pomocy języków C oraz C++. Narzędzie umożliwia między innymi zrównoleglić oraz optymalizować algorytmy napisane w C/C++ tworząc z nich bloki IP o zadeklarowanym przez użytkownika interfejsie. Natomiast Petalinux pozwala na skonfigurowanie i stworzenie systemu operacyjnego z jądrem Linux na wybraną platformę z wybranymi narzędziami i oprogramowaniem. Układ FPGA idealnie nadaje się do komunikacji z szybkimi przetwornikami, jednak stworzenie autonomicznego oscyloskopu z wykorzystaniem układu FPGA wymaga

zaimplementowania modułów odpowiedzialnych za wyświetlanie i przetwarzanie próbek oraz zaimplementowanie interfejsów HID (ang. Human Interface Device) potrzebnych do obsługi pomiarów i manipulacji danych. Natomiast wspomaganie się mikroprocesorem Microblaze oraz systemem Petalinux ułatwia implementację poszczególnych modułów, jednak jest kosztowne w zasoby, przez co wymaga wydajnego układu FPGA.

2.2.4 Płytki z układem SoC Xilinx Zynq

Zynq firmy Xilinx jest platformą SoC, która posiada zintegrowany jeden lub wiele mikroprocesorów (PS) oraz układ FPGA (PL), jako jeden układ scalony. Mikroprocesory zawarte w chipie, oparte są o architekturę ARM Cortex-A 32 lub 64-bitową, dodatkowo w serii Zynq UltraScale+ występują zintegrowane mikroprocesory czasu rzeczywistego oparte na architekturze Cortex-R, dedykowany układ graficzny oraz enkodery i dekodery wideo H.264 i H.265. Zintegrowany układ FPGA zawarty w SoC Zynq-7000 posiada architekturę zbliżoną pod względem zasobów do serii FPGA Artix-7 oraz w wyższych wariantach Kintex-7, natomiast seria Zynq UltraScale+ do serii układów Artix UltraScale+ i Kintex UltraScale+. Część procesorowa posiada dedykowane porty wejścia/wyjścia, połączone z jednostką centralną za pomocą interfejsu AMBA, natomiast struktura programowalna pozwala na rozszerzenie układu o dodatkowe komponenty, takie jak własne peryferia, akceleratory, czy też interfejsy komunikacyjne. Schemat blokowy [3] architektury układu został przedstawiony na Rysunku 3.



Rysunek 3: Architektura Zynq-7000 [3].

Za podstawową komunikację pomiędzy układem logiki a mikroprocesorem odpowiadają porty AXI, EMIO (ang. Extended Multi-Use IO), ACP (ang. Accelerator Coherency Port), kanały DMA (ang. Direct Memory Access) oraz interfejs sygnałów przerwań. Dodatkowo układy Zynq UltraScale+ posiadają dedykowane interfejsy GEM (ang. Gigabit Ethernet MAC) oraz DisplayPort. Gotowe rozwiązania wykorzystujące układy Zynq jako jednostki centralne dostępne są w formie urządzeń SBC lub SoM (ang. System on Module). Urządzenia są wyposażone zarówno w pamięć stałą typu Flash, jak i w pamięć RAM (ang. Random Access Memory), która podłączona jest do części

procesorowej i może zostać udostępniona strukturze logiki programowalnej poprzez moduły DMA. Natomiast funkcjonalność FPGA pozwala na rozszerzenie dostępnych zasobów pamięci poprzez zaimplementowanie w logice kontrolera i dołączenie pamięci w postaci kości SO-DIMM (ang. Small Outline Dual In-line Memory Module), przykładem może być płytka TySOM-3 firmy Aldec. Moc obliczeniowa, dostępne peryferia oraz zasoby pamięci w części procesorowej pozwalają na implementację, zarówno aplikacji typu bare-metal stworzonej w programie Vitis, jak i wykorzystanie pełnego systemu operacyjnego z jądrem Linux stworzonego przez narzędzie Petalinux, bez konieczności skorzystania z zasobów dołączonej logiki programowalnej. Natomiast część logiki programowalnej pozwala na implementację szybkich interfejsów komunikacyjnych z przetwornikami analogowo-cyfrowymi oraz przechowywanie, jak i wstępne przetwarzanie zebranych próbek.

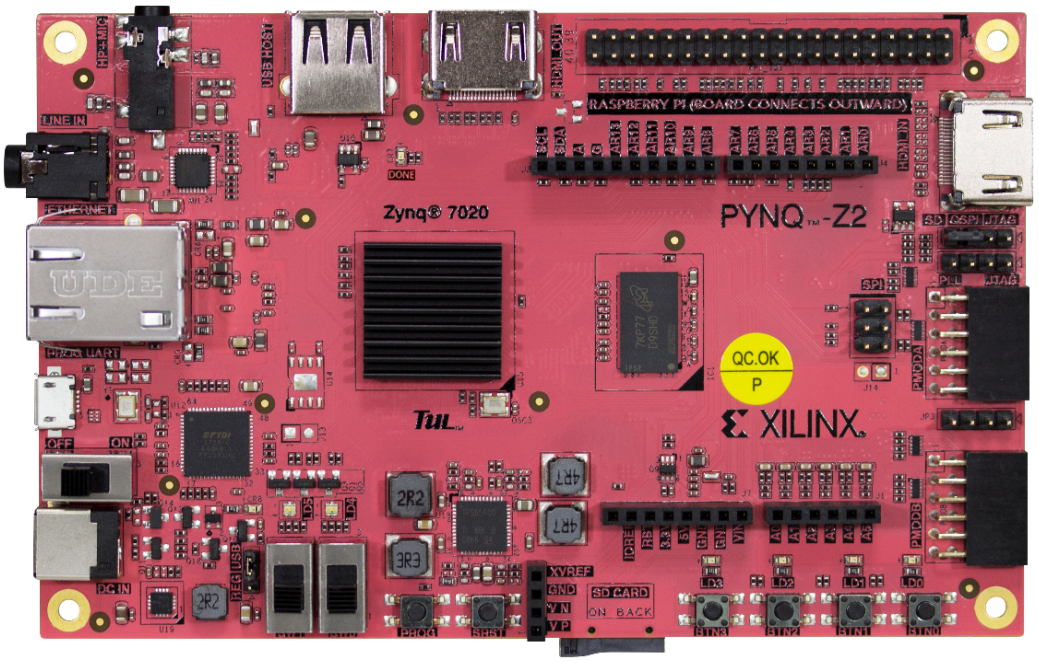
2.2.5 Podsumowanie

Analiza dostępnych na rynku urządzeń pozwoliła na wyłonienie układów SoC Zynq jako architektury, która spełnia wszystkie wymagania projektowe. Odznacza się w stosunku do mikrokontrolerów zarówno większymi zasobami pamięci, jak i większą mocą obliczeniową, dodatkowo oferując pełny system operacyjny zdolny do wyświetlania grafiki. Względem mikroprocesorów dostępnych na płytkach SBC, układ umożliwia komunikację z szybkimi przetwornikami, akcelerację obliczeń oraz pozwala w prosty sposób rozbudować urządzenia o dodatkowe peryferia. Struktura logiczna układu pozwala zapewnić ciągły przesył serii danych oraz niskie opóźnienia, które są kluczowe dla układów pomiarowych z możliwością podglądu mierzonego sygnału. Natomiast względem tradycyjnych układów FPGA, eliminuje konieczność implementacji wszystkich potrzebnych modułów wymaganych do stworzenia autonomicznego oscyloskopu, dodatkowo ceny płytek z układami Zynq są dużo niższe w porównaniu do płytek FPGA z dużymi zasobami logiki.

Architektura SoC Zynq pomimo swoich możliwości nie jest w stanie zapewnić wydajności, jaką oferują tradycyjne oscyloskopy, spowodowane jest to głównie przez opóźnienia, jakie wprowadza układ procesorowy oraz system operacyjny. System procesorowy ogólnego przeznaczenia nie jest w stanie dorównać układom typu ASIC pod względem wydajności, które zostały stworzone tylko na potrzeby wykonania ściśle określonego zadania. Opóźnienia generowane są między innymi podczas odczytu danych z pamięci, przetwarzania otrzymanych sygnałów oraz prezentacji wyników na wyświetlaczu. Jednak w porównaniu z przystawkami USB, które wykorzystują interfejs szeregowy do przesyłania danych oraz komputer PC do przetwarzania i wyświetlania sygnału, implementacja na układzie SoC posiada wyraźnie mniejsze opóźnienia. Natomiast w przypadku pomiarów w trybie pojedynczego wyzwolenia, generowane opóźnienia nie stanowią problemu, dopóki cały mechanizm wyzwalania znajduje się w strukturze programowalnej układu, a także pamięć próbek po stronie układu FPGA jest dostatecznie duża, aby skompensować opóźnienia systemu procesorowego. System procesorowy natomiast eliminuje konieczność implementacji wszystkich potrzebnych modułów wymaganych do stworzenia autonomicznego oscyloskopu przy użyciu układu wyposażonego tylko w strukturę logiczną. Dodatkowo ceny płytek z układami Zynq są dużo niższe w porównaniu do płytek FPGA z dużymi zasobami logiki. Spośród dostępnych na rynku płytek rozwojowych, zostały wybrane płytki PYNQ-Z2 firmy TUL oraz Ultra96-V2 firmy Avnet, które posłużą w projekcie do wykonania potrzebnych badań oraz porównania wydajności architektur.

Jednostką centralną PYNQ-Z2 jest układ Zynq XC7Z020-1CLG400C należący do serii Zynq-7000, wyposażony w dwurdzeniowy mikroprocesor ARM Cortex-A9 o taktowaniu 650 MHz ze wbudowanym kontrolerem pamięci DDR3, zawierającym osiem kanałów DMA i cztery porty AXI3 wysokiej wydajności. Logika programowalna posiada 85 tysięcy komórek logicznych, 220 jednostek DSP, 4.9 Mb pamięci BRAM, cztery bloki zarządzania zegarem oraz zintegrowany przetwornik analogowo-cyfrowy XADC. Na płytce znajduje się 512 MB pamięci RAM DDR3, dostępnej dla mikroprocesora oraz 16 MB pamięci flash na system i

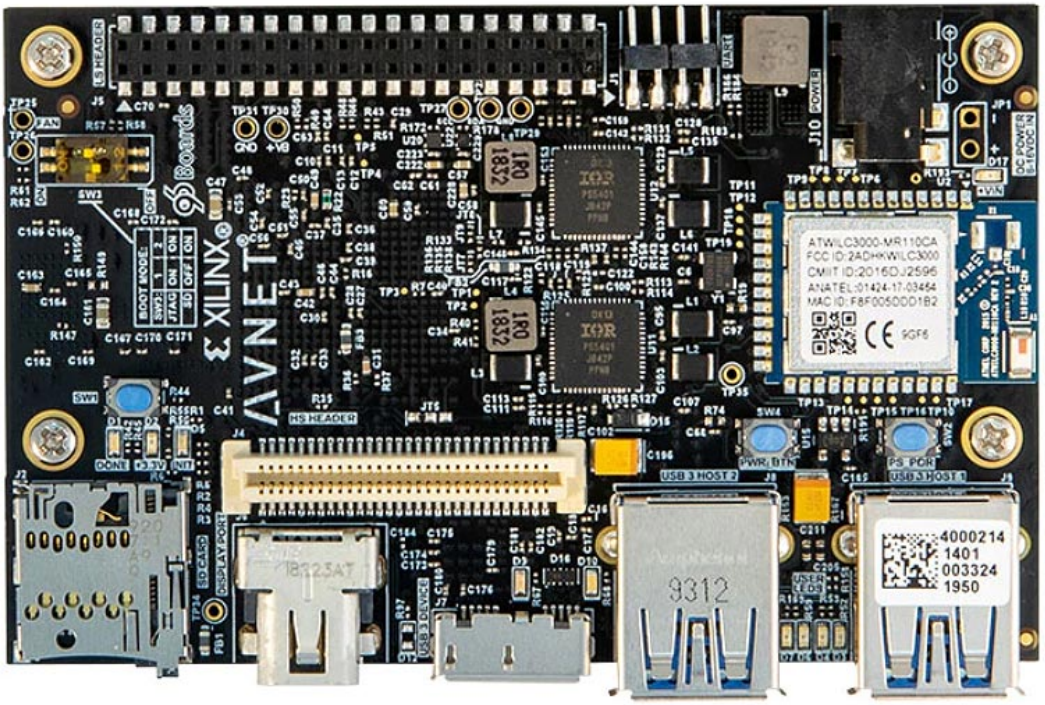
dane. Dodatkowo płytka posiada złącza MicroSD, USB 2.0 typu A, Ethernet, HDMI i audio jack, w obu przypadkach jest to zarówno wejście, jak i wyjście. Do programowania i debugowania układu służy wejście USB typu Micro-B, wyposażone w interfejs JTAG oraz UART. Do interakcji z układem dostępne są cztery przyciski, dwa przełączniki, cztery diody LED oraz dwie diody RGB LED. Urządzenie pozwala na podłączenie dodatkowych peryferiów poprzez złącze w standardzie Raspberry Pi, Arduino oraz dwa złącza PMOD (ang. peripheral module). Płytka może być zasilana poprzez standardowe złącze DC 2,5 mm lub poprzez złącze USB Micro-B programatora. Zdjęcie poglądowe płyty [4] przedstawione jest na Rysunku 4.



Rysunek 4: Płytka PYNQ-Z2 [4].

Płytka Ultra96-V2 została wyposażona w układ XCZU3EG-1SBVA484I należący do serii Zynq UltraScale+, posiadający czterordzeniową jednostkę centralną ARM Cortex-A53, dwurdzeniową jednostkę ARM Cortex-R5 oraz układ graficzny Mali-400. Płytka posiada również 2 GB pamięci RAM DDR4, złącze MicroSD, dwa porty USB 3.0 typu A połączone z dwoma hubami w trybie host, jeden port USB 3.0 Micro-B w trybie „device”, złącze wideo Mini DisplayPort, 4-pinowe

złącze UART oraz 8-pinowe złącze JTAG służące do programowania i debugowania płytki. Na płytce został umieszczony również układ WiFi oraz Bluetooth ATWILC3000 firmy Microchip do komunikacji bezprzewodowej. Do dyspozycji użytkownika są cztery diody LED oraz jeden przycisk. Natomiast podłączenie dodatkowych układów możliwe jest poprzez złącza w standardzie 96boards, złącze 40-pinowe przeznaczone jest do komunikacji z peryferiami o wolnych interfejsach oraz 60-pinowe odpowiedzialne za podłączenie szybkich interfejsów, takich jak na przykład USB, czy Gigabit Ethernet. Płytka wymaga również zasilania o napięciu 12 V i natężeniu prądu co najmniej 2 A, które doprowadzone zostaje poprzez złącze DC 1,7 mm. Zdjęcie poglądowe płyty [5] przedstawione jest na Rysunku 5.



Rysunek 5: Płytka Ultra96-V2 [5].

Rozdział III

Projekt części sprzętowej oscyloskopu

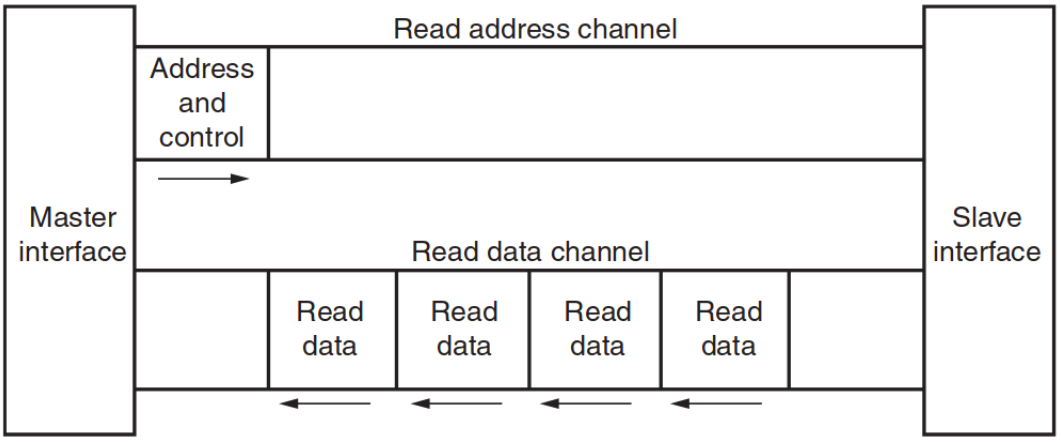
Rozdział zawiera informacje związane z implementacją bloków podstawowych systemu, bloków pomiarowych oraz analizę interfejsów komunikacyjnych dostępnych pomiędzy układem logiki programowalnej a systemem procesorowym. W rozdziale zostały przeanalizowane protokoły AMBA AXI, a następnie na podstawie analizy zostały stworzone bloki pomiarowe oparte o opisywane interfejsy.

3.1 Analiza interfejsów dostępnych w układzie SoC

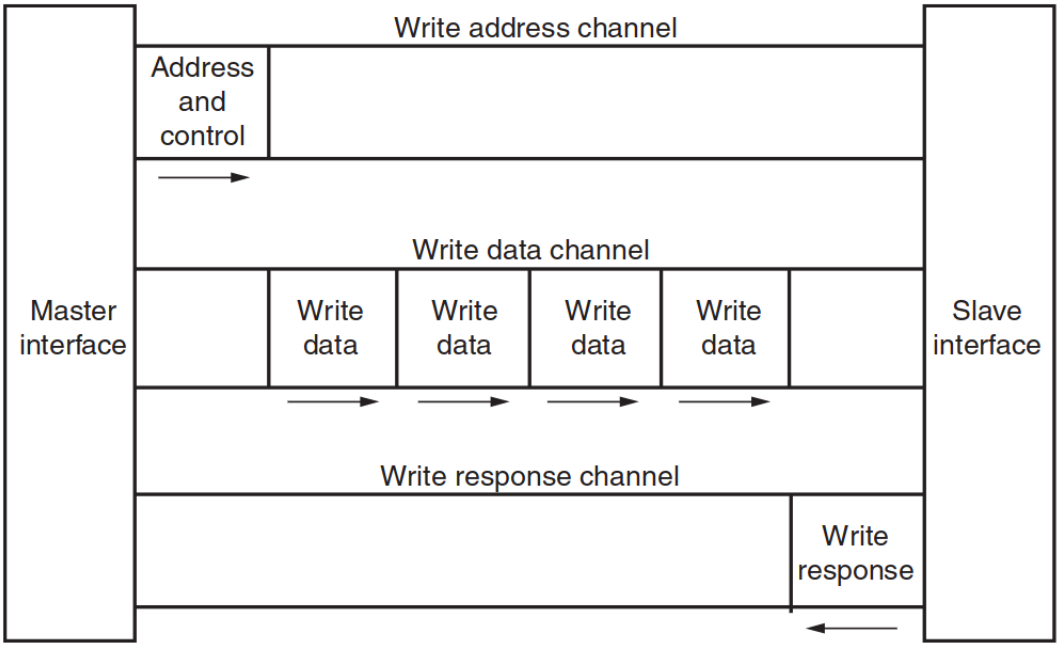
3.1.1 Interfejs AXI4

Specyfikacja protokołów AXI, AXI-Lite oraz AXI-Stream [6] powstała w 1996 roku jako część architektury magistrali AMBA firmy ARM, a oficjalnie została wydana w 2003 roku pod nazwą AXI3 wraz z wprowadzeniem trzeciej wersji specyfikacji AMBA, natomiast od 2010 roku funkcjonuje w wersji czwartej jako AXI4. Protokół AXI4 określany jest też jako pełne AXI (ang. AXI4 Full) lub AXI4 MM (ang. Memory Mapped), oferuje wysokie przepustowości transferu danych o niskich opóźnieniach, pozwala na bezpośrednią komunikację pomiędzy urządzeniami oraz pracę w trybie współdzielonej magistrali poprzez moduły AXI Interconnect. Interfejs wykorzystuje pięć kanałów do komunikacji pomiędzy urządzeniami: kanał adresowy transakcji odczytu, kanał adresowy transakcji zapisu, kanał danych transakcji odczytu, kanał danych transakcji zapisu oraz kanał danych odpowiedzi na transakcję zapisu, które pozwalają na równoczesną komunikację w obu kierunkach. Zarówno AXI, jak i AXI-Lite bazują na wymianie danych z wykorzystaniem adresu pamięci, pod którym następnie dane zostaną wpisane lub z którego dane zostaną odczytane. Podczas każdej transakcji, do której dochodzi przy użyciu danych protokołów, najpierw musi dojść do

potwierdzenia adresu oraz dodatkowych sygnałów sterujących, zanim dojdzie do transferu danych. Protokół pozwala na seryjny (ang. burst) transfer danych podczas pojedynczej transakcji, oferując maksymalnie do 256 transferów danych, w takim przypadku dane, które znajdują się w obszarze ciągłym pamięci, mogą zostać odczytane podczas jednej transakcji z użyciem adresu bazowego. Koniec transakcji z seryjnym transferem sygnalizowany jest sygnałem „TLAST". Przebieg transakcji odczytu danych został przedstawiony na Rysunku 6, natomiast transakcji zapisu i odpowiedzi na zapis na Rysunku 7 [7].

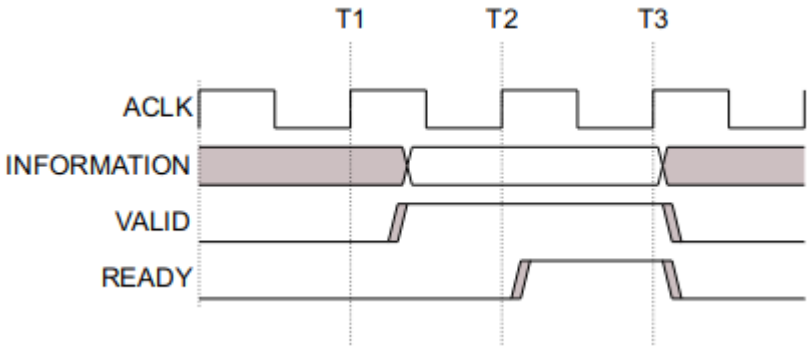


Rysunek 6: Przebieg transmisji odczytu dla interfejsu AXI [7].

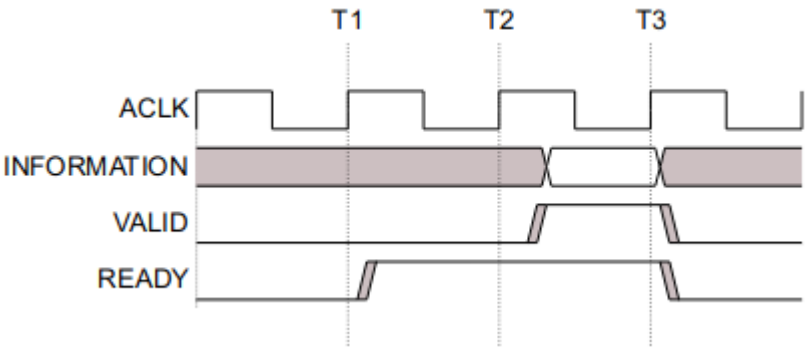


Rysunek 7: Przebieg transmisji zapisu i odpowiedzi na zapis dla interfejsu AXI [7].

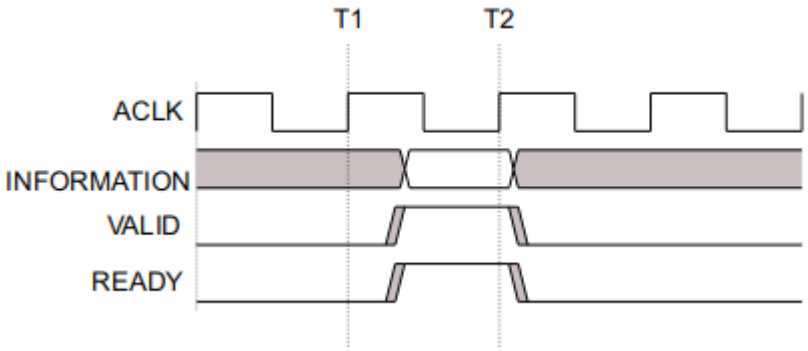
Każda transakcja na dowolnym kanale, która zachodzi z użyciem interfejsów AXI, poprzedzona zostaje procesem „handshake”, który na podstawie stanów sygnałów „TREADY” oraz „TVALID” decyduje o rozpoczęciu przesyłu danych. Transfer zostaje rozpoczęty tylko wtedy, gdy obie linie sygnałowe są ustawione, domyślnie jest to wysoki stan logiczny, na przynajmniej jeden cykl zegara. Dodatkowo sygnały posiadają ściśle określone zależności. Sygnał „TVALID”, który został raz ustawiony, nie może zmienić swojego stanu, dopóki nie dojdzie do wymiany danych, to znaczy sygnał „TREADY” nie zostanie ustawiony lub nie wystąpi reset układu. Natomiast sygnał „TREADY” może zmienić swój stan, nawet zanim dojdzie do rozpoczęcia transakcji.



Rysunek 8: Przebieg procesu handshake, gdy sygnał „TVALID” został ustawiony jako pierwszy [6].

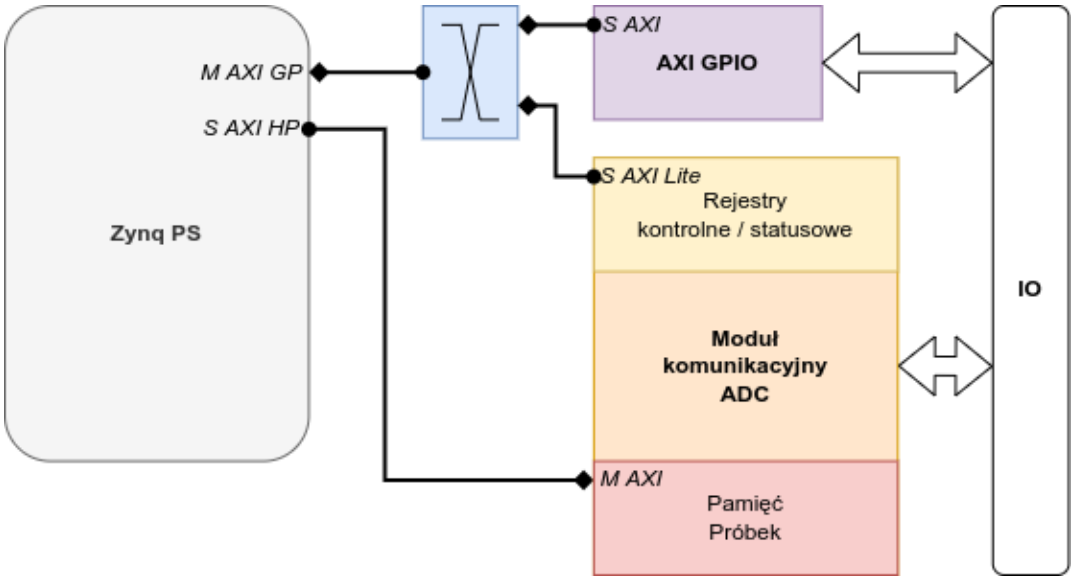


Rysunek 9: Przebieg procesu handshake, gdy sygnał „TREADY” został ustawiony jako pierwszy [6].



Rysunek 10: Przebieg procesu handshake, gdy oba sygnały zostały ustawione równocześnie [6].

Układ XC7Z020, w który została wyposażona płytka PYNQ-Z2, ma cztery porty AXI4 ogólnego przeznaczenia (AXI GP) o 32-bitowej szynie danych, gdzie dwa są typu „Master", dwa typu „Slave". Dodatkowo układ posiada cztery porty typu „Slave" wysokiej wydajności (AXI HP) o 32 lub 64-bitowej szerokości szyny danych. Natomiast układ XCZU3EG na płytce Ultra96-V2 posiada tylko porty wysokiej wydajności, gdzie trzy są typu „Master" oraz siedem typu „Slave", oraz oferują 32, 64 lub 128-bitową szynę danych. Porty wysokiej wydajności zapewniają wysoką przepustowość danych poprzez bezpośredni dostęp do systemu pamięci znajdującego się w części procesorowej. W projekcie porty AXI HP posłużą do przesyłu danych pomiędzy częścią procesorową a strukturą logiki programowalnej w celu zapewnienia najwyższej przepustowości danych. Natomiast porty AXI GP zostaną wykorzystane sterowania modułu odpowiedzialnego za zbieranie próbek oraz do odczytu stanów przełączników znajdujących się na płytce poprzez moduł AXI GPIO. Rysunek 11 przedstawia przybliżony schemat blokowy układu.



Rysunek 11: Schemat blokowy układu z wykorzystaniem interfejsu AXI.

3.1.2 Interfejs AXI4-Lite

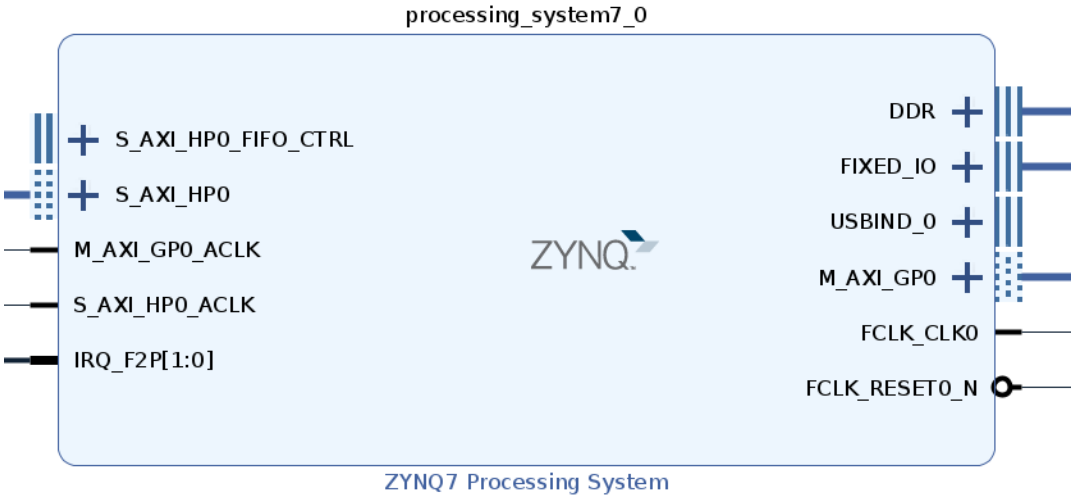
Protokół AXI-Lite powstał jako uproszczona wersja interfejsu AXI, względem oryginału zachował pięć kanałów komunikacyjnych, dwukierunkowość oraz kompatybilność z urządzeniami AXI. Natomiast została zmieniona szerokość magistrali danych do wartości 32 i 64-bity, dodatkowo została usunięta funkcjonalność seryjnych transakcji oraz związane z nią sygnały. Protokół powstał na potrzeby dostępu do rejestrów kontrolnych i statusowych urządzeń oraz wymiany danych z prostymi modułami, które nie wymagają dużych lub szybkich transferów danych. W projekcie zostanie wykorzystany tylko do zarządzania modułami i podglądu rejestrów statusowych.

3.2 Implementacja bloków podstawowych

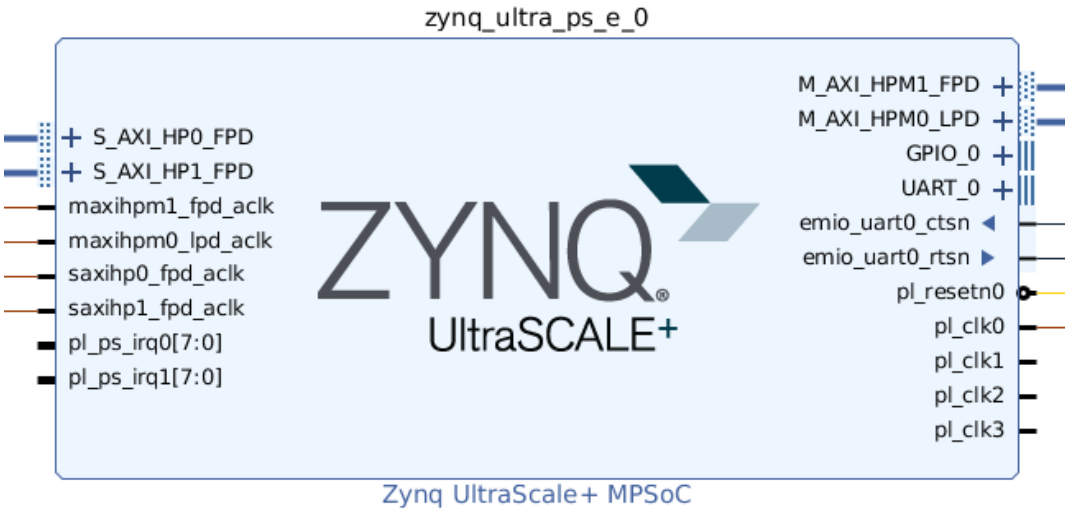
Firma Xilinx, która jest producentem architektury Zynq, udostępnia kompletny zestaw narzędzi oraz rozwiązań potrzebnych do stworzenia oprogramowania na ich platformy. W pierwszym etapie przygotowania części sprzętowej należy przygotować sprzętową architekturę układu z wykorzystaniem narzędzia Vivado. Środowisko zostało zainstalowane w wersji 2020.2, które jest obecnie wspierane przez nakładkę PYNQ. Podstawową funkcjonalnością programu Vivado jest

synteza stworzonej architektury, implementacja na dane urządzenie oraz generację pliku bitstream dla danego układu logiki. Dodatkowo program umożliwia symulację napisanego kodu, co pozwala sprawdzić implementowane funkcje i ich wyniki, zanim zostaną wgrane do urządzenia. Analiza zaimplementowanej funkcjonalności na działającym układzie również jest dostępna i realizowana jest poprzez wewnętrzny analizator stanów z użyciem interfejsu JTAG. Program posiada podsystem pozwalający na stworzenie całej architektury układu poprzez interfejs graficzny w postaci schematu blokowego przy użyciu dostępnych bloków IP. Za pomocą tego narzędzia zostaną stworzone oraz zintegrowane podstawowe struktury układu, które zostaną wykorzystane w późniejszych etapach do skonfigurowania narzędzia Petalinux oraz PYNQ.

Podstawowym elementem struktury układu jest system procesorowy, za którego konfigurację odpowiada odpowiednio blok IP „ZYNQ7 Processing System” dla układu XC7Z020, który został przedstawiony na Rysunku 13 oraz „Zynq UltraScale+ MPSoC” dla układu XCZU3EG, który z kolei został przedstawiony na Rysunku 14.

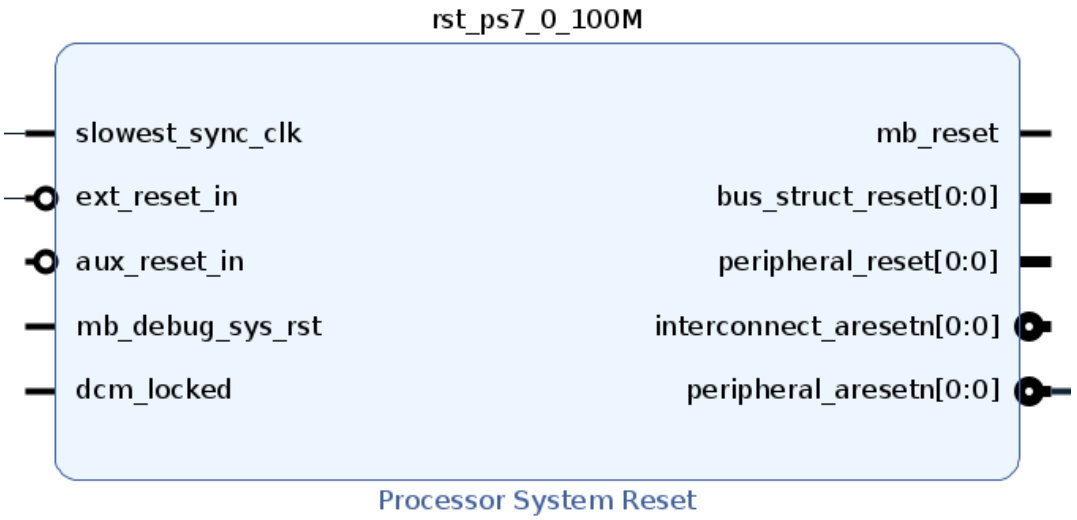


Rysunek 13: Blok konfiguracyjny systemu procesorowego Zynq-7000.



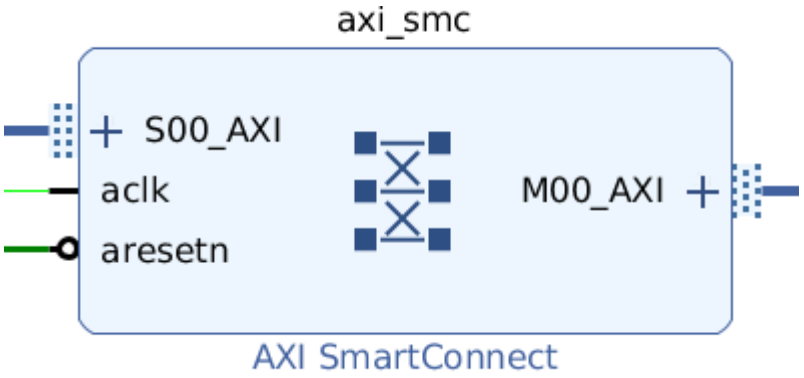
Rysunek 14: Blok konfiguracyjny systemu procesorowego Zynq UltraScale+ MPSoC.

Blok zostanie wykorzystany do skonfigurowania częstotliwości zegarów pamięci, procesora, logiki oraz układów peryferyjnych, kontrolera pamięci, układów peryferyjnych, przerwań oraz interfejsów służących do komunikacji ze strukturą logiki. Podstawowa konfiguracja w przypadku tego elementu zostaje ustawiona automatycznie po uruchomieniu zintegrowanej funkcji Vivado „Run Block Automation". Funkcja ustawia domyślne parametry zegarów, peryferiów oraz pamięci na podstawie plików konfiguracyjnych dostarczonych przez producenta płytki. Po zakończeniu automatycznej konfiguracji ręcznie zostały ustawione wartości zegarów logiki na częstotliwość 200 MHz, zostały włączone porty AXI Slave wysokiej wydajności, porty AXI Master ogólnego przeznaczenia oraz porty przerwań od strony logiki programowalnej. Następnym podstawowym elementem układu jest blok „Procesor System Reset" odpowiedzialny za generację sygnału resetującego dla podłączonych bloków IP w danej domenie zegarowej na podstawie zewnętrznego sygnału reset. Moduł pozwala na skonfigurowanie zarówno dodatkowe sygnały wyjściowe reset, jak i źródła sygnału reset oraz ich poziomów logicznych. W projekcie sygnał resetu generowany jest przez system procesorowy układu Zynq, natomiast moduły resetowane są w sposób synchroniczny, przy pomocy aktywnego nisko sygnału reset. Rysunek 15 przedstawia dany blok dla domeny zegarowej o częstotliwości 100 MHz.

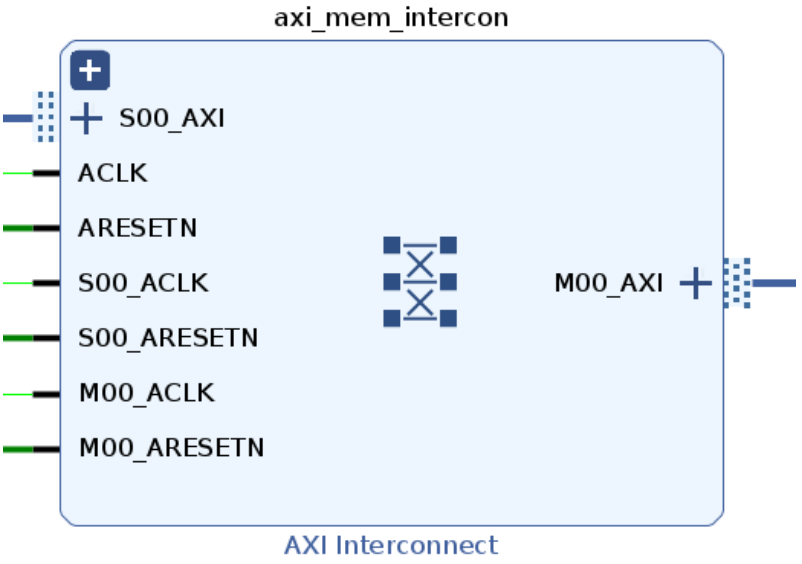


Rysunek 15: Blok konfiguracyjny odpowiedzialny za system sygnałów resetujących.

Natomiast za łączenie bloków IP z interfejsem AXI lub AXI-Lite odpowiadają moduły AXI Interconnect oraz AXI SmartConnect, które przedstawione są na Rysunku 16 i Rysunku 17. Układ pozwala na konwersje sygnałów w standardzie AXI4 na AXI3, dynamiczną zamianę szerokości szyny danych oraz na łączenie modułów o różnych domenach zegarowych. Połączenia pomiędzy układami mogą zostać poprowadzone ręcznie lub automatycznie podczas wykonywania funkcji „Run Connection Automation". Należy również pamiętać o ustawieniu odpowiednich adresów pamięci dla modułów z interfejsem AXI lub AXI-Lite w zakładce „Address Editor".



Rysunek 16: Blok konfiguracyjny AXI SmartConnect.



Rysunek 17: Blok konfiguracyjny AXI Interconnect.

3.3 Implementacja bloków pomiarowych

3.3.1 Układ z interfejsem równoległym

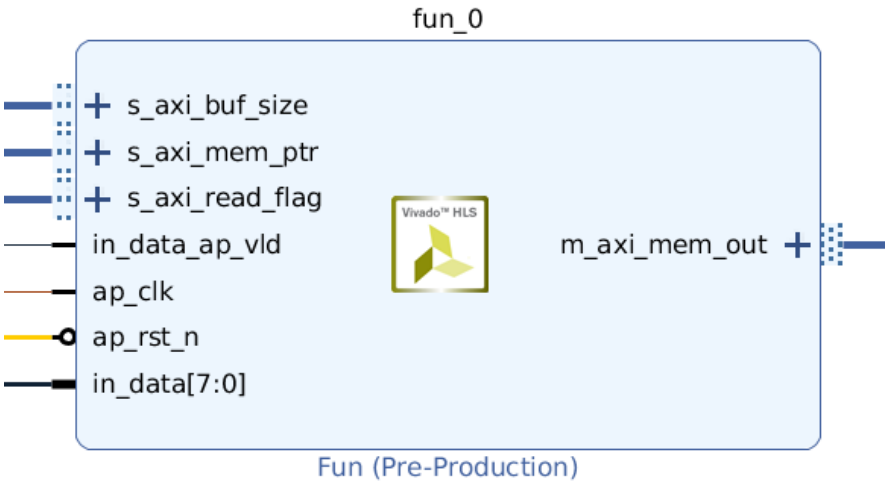
Na podstawie analizy dostępnych na rynku układów przetworników analogowo-cyfrowych uzyskano następujący wniosek, że układy o częstotliwości próbkowania 100 MSPS i więcej wykorzystują dla wyjść danych:

- interfejsy równoległe,
- interfejsy JESD204,
- szeregowe LVDS (ang. Low-voltage differential signaling).

Interfejs JESD204 [8] oferuje bardzo wysokie szybkości przesyłu danych (data rate) dla przesyłu szeregowego, które osiągają nawet 32 Gbps (ang. Gigabits per second) dla standardu JESD204C. Interfejs posiada gotowe bloki IP, które odpowiadają za deserializację, serializację i synchronizację danych oraz konwersję na interfejs dostępny w danej architekturze, na przykład dla układów firmy Xilinx jest to AXI4-Stream. W przypadku interfejsów szeregowych wymagane jest stworzenie odpowiedniego bloku konwertującego sygnały na podstawie noty katalogowej dla danego układu. Dodatkowo interfejsy szeregowe dla wysokich częstotliwości próbkowań wymagają zastosowania do transmisji danych par

różnicowych, które pozwalają na stabilny przesył przy wysokich szybkościach transmisji. Wartość szybkości przesyłu danych dla szeregowego interfejsu zazwyczaj jest wartością maksymalną częstotliwości próbkowania przemnożoną przez rozdzielczość danego przetwornika.

Interfejsy równoległe są znacznie częściej wykorzystywane w przetwornikach z częstotliwością próbkowania powyżej 100 MSPS, na przykład dla przetworników firmy Texas Instruments suma układów z interfejsem szeregowym wynosi 28, natomiast z interfejsem równoległym 122. Dodatkowo interfejsy równoległe odznaczają się dużo niższymi częstotliwościami dla wyjść danych, ze względu na możliwość wystawienia części lub całości wartości próbki w ciągu jednego taktu zegarowego Na potrzeby projektu, został stworzony układ z równoległym interfejsem komunikacyjnym, którego implementacja pozwoliła na sprawdzenie możliwości, jakie oferują narzędzia syntezy wysokiego poziomu firmy Xilinx. Moduł konwertujący dane z interfejsu równoległego na interfejs AXI, dla 8-bitowego przetwornika ADC został stworzony z wykorzystaniem narzędzia Vitis HLS, które pozwala na syntezę języka C/C++ oraz do wersji programu 2020.1 również języka SystemC.



Rysunek 18: Blok konwertujący magistralę równoległą do magistrali AXI.

Moduł posiada na wejściu 8-bitową szynę danych, sygnał zegarowy, sygnał resetujący oraz trzy interfejsy AXI4-Lite do sterowania układem, natomiast na wyjściu posiada interfejs AXI4. Wygenerowany układ został przedstawiony na Rysunku 18, natomiast kod źródłowy przedstawiony został na Rysunku 19.

```
#include "funkcja.h"
#include "string.h"

void fun(char in_data, uint32_t *out_data_p, bool rd, uint16_t buffsize){
#pragma HLS INTERFACE ap_vld port=in_data
#pragma HLS INTERFACE m_axi port=out_data_p depth=16384 bundle=mem_out
#pragma HLS INTERFACE s_axilite port=rd bundle=read_flag
#pragma HLS INTERFACE s_axilite port=buffsize bundle=buf_size
#pragma HLS INTERFACE s_axilite port=out_data_p bundle=mem_ptr
#pragma HLS INTERFACE ap_ctrl_none port=return

    static uint32_t out_data_temp[N];

#pragma HLS BIND_STORAGE variable=out_data type=ram_t2p impl=bram

    static uint16_t word = 0;
    static uint32_t iter = 0;
    static uint8_t data_pos = 0;
    static uint8_t word_arr[4];

    data_pos = iter % 4;
    word_arr[data_pos] = in_data;
    if (data_pos == 3) {
        memcpy(&out_data_temp[word], word_arr, sizeof(word_arr));
        word++;
    }

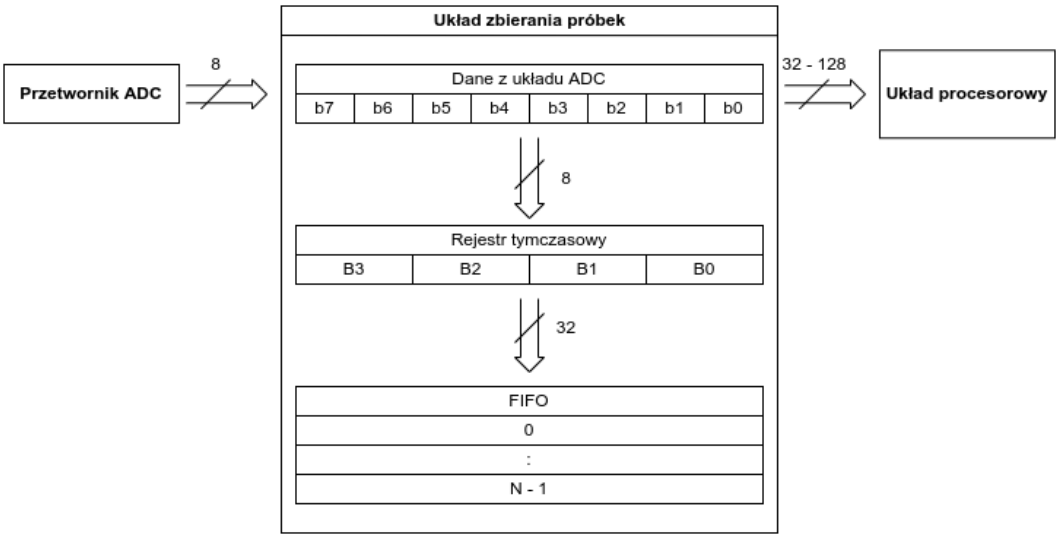
    if(word == buffsize) word = 0;
    else iter++;

if(rd == 1)
    memcpy(out_data_p, out_data_temp,sizeof(out_data_temp));

}
```

Rysunek 19: Kod źródłowy dla bloku IP z równoległym interfejsem.

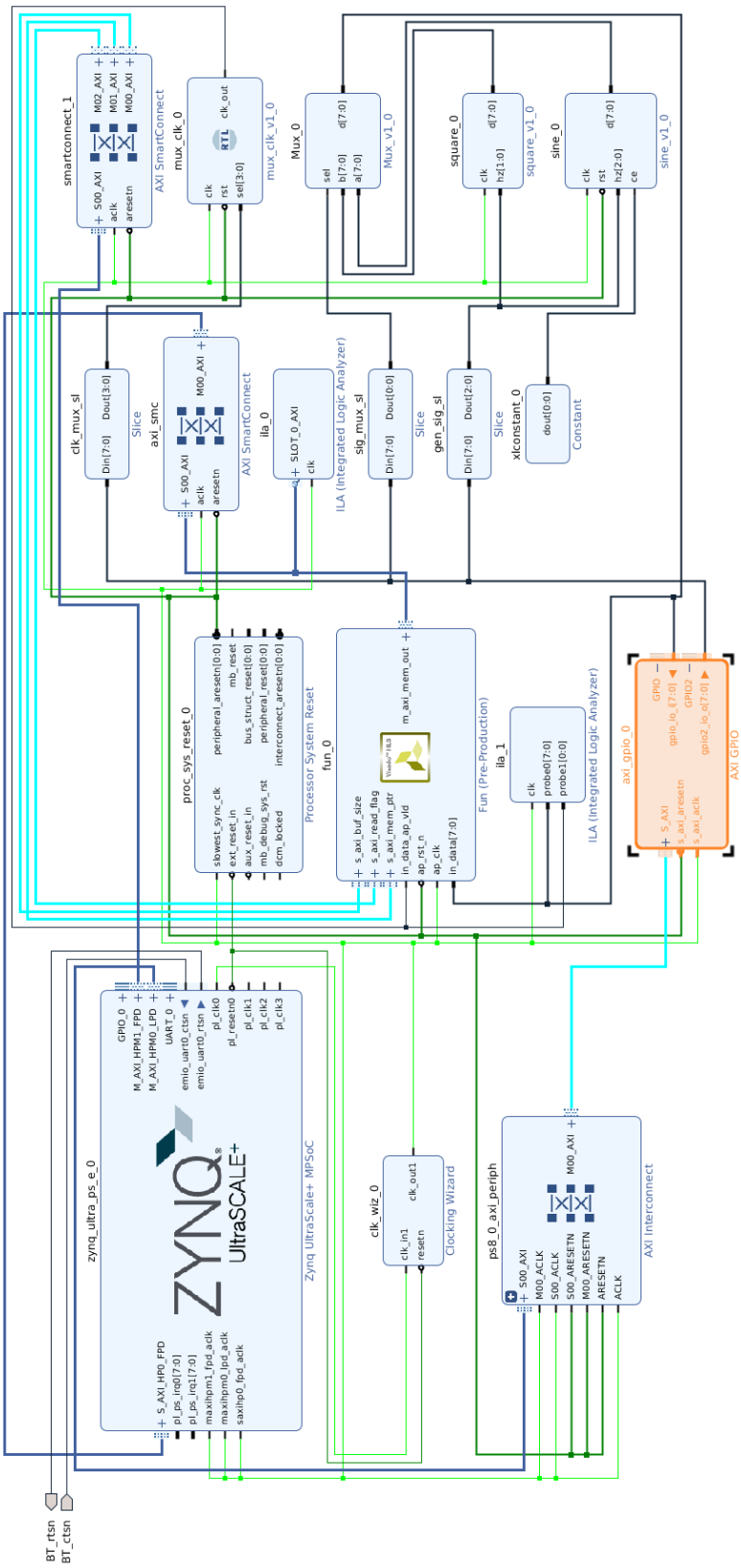
Próbki otrzymane poprzez interfejs równoległy w postaci 8-bitowego słowa, zostają wpisane do wewnętrznego rejestru, który składa je w 32-bitowe słowo, a następnie przekazuje do pamięci FIFO zadeklarowanej wewnątrz układu. Schemat blokowy został przedstawiony na Rysunku 20.



Rysunek 20: Schemat blokowy układu z równoległym interfejsem.

Moduł pozwala na odczyt próbek z przetwornika w trybie SDR (ang. Single Data Rate) z częstotliwością 400 MHz dla układu Zynq-7000, natomiast dla układu Zynq UltraScale+ wynosi 600 MHz. Interfejs AXI musi zapewnić minimalną częstotliwość odczytu próbek z pamięci FIFO o wartości odpowiednio 100 MHz oraz 150 MHz dla 32-bitowych słów, aby uniknąć przepełnienia pamięci FIFO. Natomiast istnieje również możliwość zmiany szerokości słowa dla interfejsu AXI, na wartość 64-bitową oraz dla układu Zynq UltraScale+ dodatkowo 128-bitową, co pozwala na obniżenie wymaganą minimalna częstotliwość. Narzędzie Vitis HLS [9] posiada jednak ograniczenie względem domen zegarowych dla kodów napisanych w języku C/C++, które zezwala tylko na taktowanie modułu jednym sygnałem zegarowym. Ze względu na to ograniczenie zegar współdzielony jest z interfejsem AXI. Zarówno układ Zynq-7000, jak i układ Zynq UltraScale+ wykorzystane w projekcie posiadają ograniczenie częstotliwości dla interfejsu AXI HP o wartości odpowiednio 200 MHz oraz 333 MHz. Blok IP „AXI Interconnect” posiada funkcjonalność zmiany domen zegarowych, która umożliwia komunikację układów taktowanych różnymi częstotliwościami zegara, natomiast skorzystanie z tej funkcjonalności w większości przypadków wymaga manualnej korekcji ograniczeń czasowych. Na potrzeby projektu częstotliwości zegarów zostały odpowiednio zmniejszone, aby

umożliwić podłączenie modułów do systemu procesorowego. Jako sygnały testowe zostały wykorzystane sygnał sinusoidalny, który jest generowany poprzez zapętlony odczyt próbek z pamięci oraz prostokątny o wypełnieniu 50%. Wybór sygnału realizowany jest poprzez multiplekser, do którego zostały podłączone układy. Dodatkowo konfigurowalna jest częstotliwość sygnałów testowych poprzez dzielnik sygnału zegarowego. Całość sterowana jest przez moduł AXI GPIO. Natomiast do weryfikacji sygnału mierzonego posłużył moduł „System ILA" podłączony do wyjścia i wejścia układu konwertującego. Pełny układ przedstawiony został na Rysunku 21.

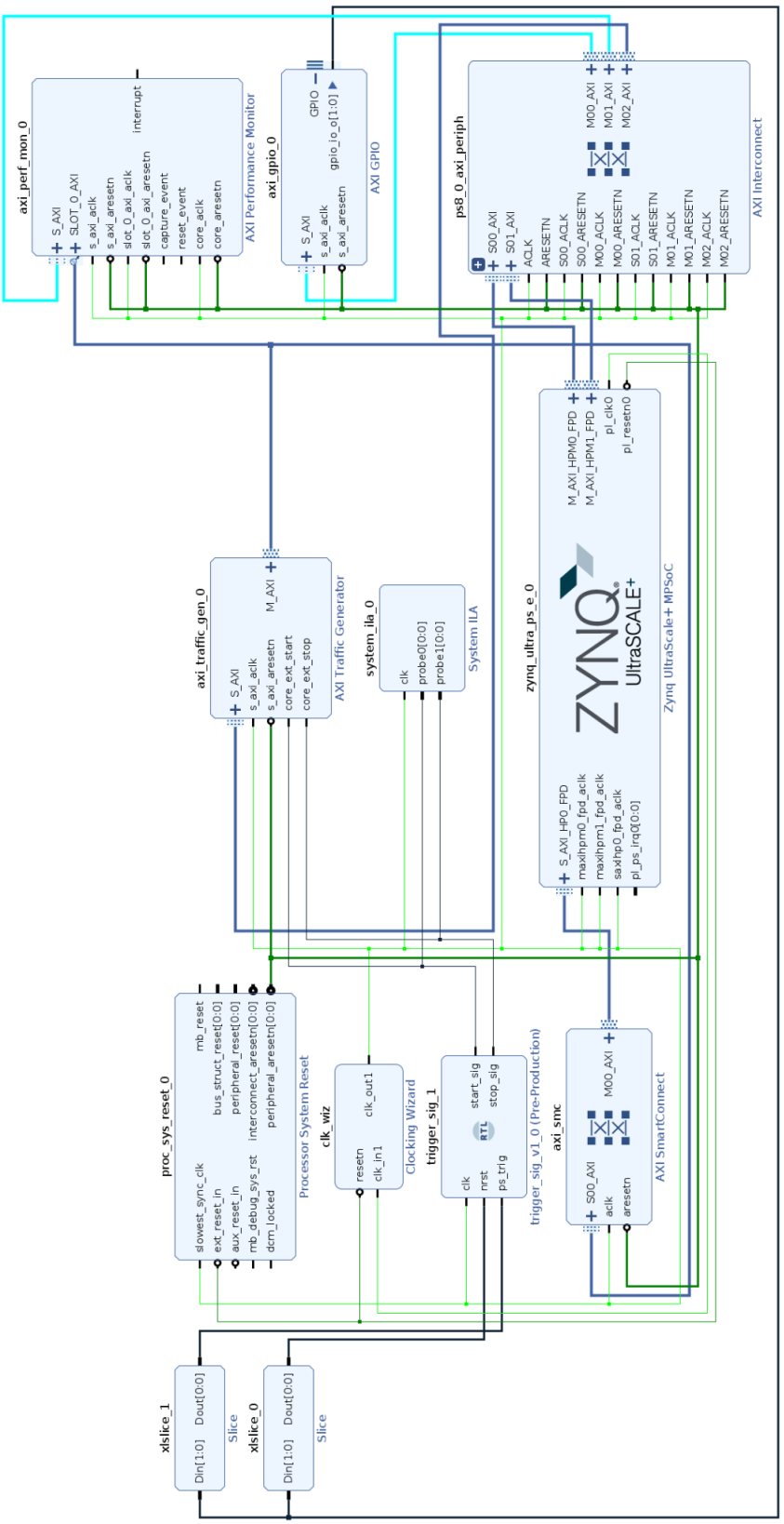


Rysunek 21: Schemat blokowy układu z interfejsem równoległym.

3.3.2 Układ z interfejsem AXI

Interfejs AXI został wykorzystany w [3.3.1], natomiast ze względu na ograniczenia zegarów interfejsów AXI systemu procesorowego, nie ukazał pełnych możliwości, jakie oferuje struktura logiki programowalnej. Na potrzeby projektu został stworzony układ testujący maksymalną przepustowość interfejsu AXI, w tym celu został wykorzystany blok IP generatora pakietów AXI. Dla układu Zynq-7000 zastała ustawiona częstotliwość 200 MHz oraz 64-bitowa szerokość szyny danych. Natomiast dla układu Zynq UltraScale+ zostały ustawione odpowiednio sygnał zegarowy 333 MHz oraz 128-bitowa szerokość szyny danych. Schemat blokowy układu został przedstawiony na Rysunku 22.

Przepustowości danych oraz stabilności interfejsu AXI została sprawdzona na podstawie ilości ukończonych transakcji w określonym czasie. Do testu został wykorzystany blok IP generatora pakietów udostępniony przez firmę Xilinx. Moduł symuluje ramki protokołów USB, PCIe, Ethernet, wideo lub wykonuje transakcje odczytu i zapisu na danych zdefiniowanych przez użytkownika. Na potrzeby testu został wykorzystany profil „Custom” w trybie Static, który pozwala na ustawienie parametrów interfejsu AXI takich, jak szerokości szyn danych, szerokość szyny adresowe, adresy pamięci, ilość pakietów wysłanych w jednej transakcji oraz wybrać kanały transmisyjne AXI, które zostaną poddane testom. Parametry profilu zostały dostosowane tak, aby wykorzystać maksymalne dostępne dla platformy parametry protokołu. Do pomiaru wydajności modułu został wykorzystany blok IP „AXI Performance Monitor”, którym została zmierzona liczba transakcji w określonym czasie. Dodatkowo na potrzeby testu został stworzony w logice programowalnej układ odpowiadający za generowanie odstępu czasowego pomiędzy wystawieniem sygnału start, rozpoczynającego generację pakietów a sygnałem stop, który zatrzymuje układ. Odstęp czasowy generowany w układzie logiki programowalnej jest znacznie dokładniejszy niż w przypadku sygnału generowanego w aplikacji przez układ procesorowy.



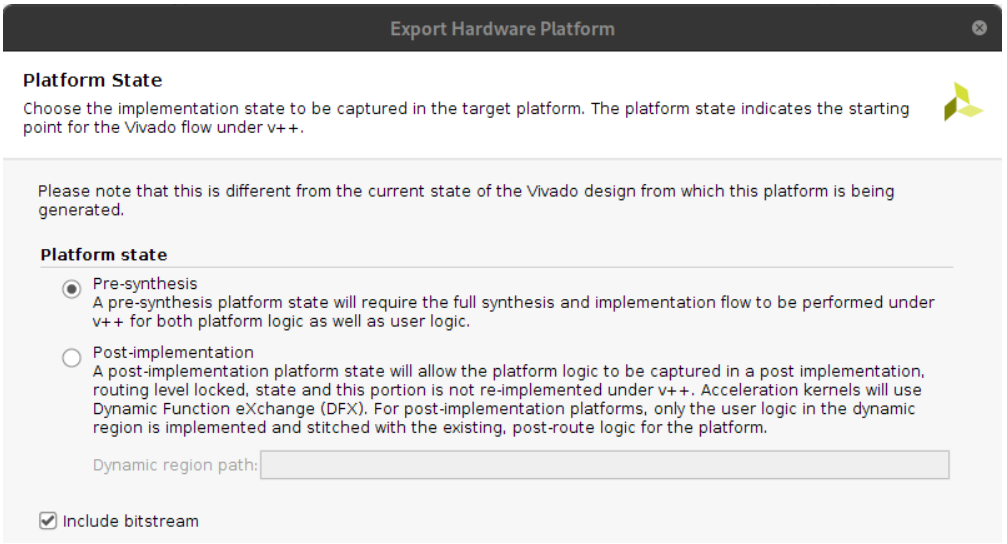
Rysunek 22: Schemat blokowy układu do pomiaru wydajności interfejsu AXI4.

3.4 Przygotowanie plików projektu Vivado

Ostatnim etapem związanym z częścią sprzętową projektu była generacja potrzebnych plików do zaprogramowania układów oraz konfiguracji środowiska PYNQ. Dla każdego z przygotowanych projektów w postaci schematów blokowy architektury należało stworzyć plik „HDL wrapper" obudowujący zdefiniowane w projekcie porty. W celu wygenerowania pliku należało w oknie plików źródłowych projektu zaznaczyć plik schematu z rozszerzeniem „.bd" oraz wybrać opcję „Create HDL wrapper". Stworzenie pliku pozwala na syntezę platformy oraz generację plików bitstream. Plik bitstream został stworzony przy pomocy funkcji Vivado „Generate Bitstream" oraz został zapisany przez program w folderze ze ścieżką: „ścieżka/do/projektu/nazwa_projektu/nazwa_projektu.runs/impl_1", natomiast plik „.hwh" został wygenerowany podczas syntezy bloku architektury w folderze:
„ścieżka/do/projektu/nazwa_projektu/nazwa_projektu.srcs/sources_1/bd/nazwa_pliku_bd/hw_handoff". Vivado pozwala wygenerować również plik XSA, który zawiera oba pliki. W celu wygenerowania pliku XSA należy wybrać odpowiednio „File->Export Hardware", następnie w oknie należy wybrać typ „Expendable", jak zostało przedstawione na Rysunku 23, w kolejnym oknie wybrać stan platformy na „Pre-synthesis" oraz zaznaczyć pole wyboru „Include Bitstream" tak jak na Rysunku 24. W kolejnym oknie program pozwoli ustawić podstawowe informacje, takie jak: nazwa platformy, wersja, dostawca, nazwa płytki oraz krótki opis eksportowanej platformy. Ostatnie okno konfiguracji pozwala nadać nazwę dla pliku XSA oraz wskazać miejsce eksportu.



Rysunek 23: Okno konfiguracyjne wyboru typu podczas eksportu platformy.



Rysunek 24: Okno konfiguracyjne wyboru stanu podczas eksportu platformy.

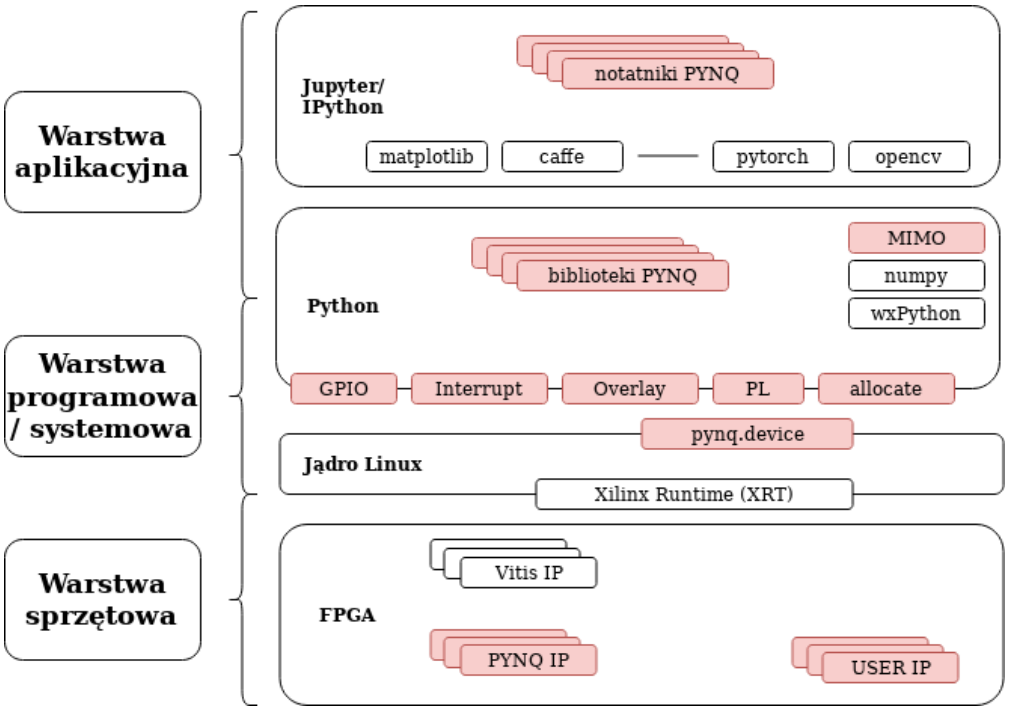
XSA jest plikiem typu skompresowanego archiwum, które zawiera w sobie wiele plików konfiguracyjnych potrzebnych do stworzenia definicji platformy wymaganej w oprogramowaniu Vitis lub Vitis HLS. Na potrzeby projektu pliki zostaną wykorzystane w późniejszym etapie do konfiguracji środowiska PYNQ dla danej platformy.

Rozdział IV

Projekt części programowej

4.1 Środowisko PYNQ

PYNQ jest otwartoźródłowym projektem firmy Xilinx [10], który oparty został o system Petalinux. Środowisko PYNQ wykorzystuje język i biblioteki Python do interakcji z blokami IP zaimplementowanymi w strukturze logiki programowalnej. Domyślnie PYNQ wykorzystuję przeglądarkę internetową oraz lokalną instancję serwera Jupyter Notebook, który pozwala na wykonywanie skryptów oraz prezentację wyników w formie interaktywnych stron. Wykorzystanie języka Python pozwala na szybsze prototypowanie i rozwój architektury w porównaniu do implementacji opartych tylko o programy napisane w języku C, natomiast środowisko umożliwia wykorzystanie programów oraz bibliotek napisanych w języku C/C++ dla aplikacji, które wymagają wyższych wydajności. Środowisko wykorzystuje pliki z rozszerzeniem „.bit” oraz „.hwh” do poprawnej interakcji z układem logiki, pliki generowane są przez oprogramowanie Vivado przy użyciu funkcji „Generate Bitstream”. Pliki wyżej wymienione zawierają konfigurację układu oraz informacje, takie jak: nazwy, wersje, interfejsy oraz adresy pamięci dla danych bloków IP, dodatkowo zawiera informacje o domenach zegarowych i sygnałach przerwań. Pliki konfiguracyjne wykorzystywane są do stworzenia w środowisku nakładki „overlay”, która w dokumentacji wymieniana jest również jako biblioteka sprzętowa, nakładka odpowiada za kontrolowanie układów w strukturze logiki programowalnej poprzez system procesorowy i kod napisany w języku Python, bez konieczności pisania i kompilacji odpowiednich sterowników oraz ładowania ich w systemie Linux. Funkcjonalności środowiska pozwalają na implementację i kontrolę złożonych architektur przez użytkowników, którzy nie posiadają zaawansowanej wiedzy o systemie operacyjnym i sterownikach. Architektura środowiska została przedstawiona na Rysunku 25.



Rysunek 25: Architektura środowiska PYNQ.

4.2 Przygotowanie środowiska PYNQ

Środowisko PYNQ przygotowane zostało na komputerze z zainstalowanym systemem Ubuntu 18.04, który zalecany jest przez deweloperów projektu do budowy i rekonfiguracji obrazów systemu ze środowiskiem PYNQ. Obraz systemu został przygotowany manualnie tylko dla płytki Ultra96-V2, ze względu na brakujący kod sterownika dla układu WiFi w jądrze Linux, który wykorzystany jest w oficjalnym obrazie. Natomiast dla płytki PYNQ-Z2 został wykorzystany oficjalny obraz dostępny na stronie projektu PYNQ. Narzędzie konfigurujące i generujące system ze środowiskiem PYNQ w wersji 2.7 wymagało zainstalowania oprogramowania Petalinux oraz Vivado HLx w wersji 2020.2, następnie po zainstalowaniu pakietów należało ściągnąć kod źródłowy środowiska z oficjalnego repozytorium Git [O1] i przełączyć się na gałąź wersji 2.7 za pomocą terminala oraz komendy „git checkout image_v2.7”. Przed przystąpieniem do konfiguracji narzędzia należało również wykonać komendy dla wcześniej pobranych pakietów:

- „source </ścieżka/do/pakietu/petalinux>/settings.sh” oraz
- „source </ścieżka/do/pakietu/Vivado>/settings.sh”,

które umożliwiają na uruchomienie danych programów w aktywnej konsoli. Następnie w folderze „</ścieżka/do/repozytorium>/PYNQ/sdbuild/scripts” należało uruchomić skrypt „setup_host.sh”, który odpowiedzialny jest za pobranie i aktualizację pakietów potrzebnych do przygotowania obrazu. Należało również pobrać definicje układu oraz potrzebne pakiety dla płytki Ultra96-V2, w tym celu wykorzystane zostało główne repozytorium Git [O2] producenta oraz gałąź wersji 2.7. W pobranym repozytorium należało uruchomić skrypt „./build96.sh 2”. Skrypt pobiera oraz konfiguruje odpowiednio projekt Petalinux, który wykorzystuje do generacji obrazu. Podczas kompilacji pakietów mogą wystąpić błędy, znaczna ich część związana jest z niedostępnymi chwilowo repozytoriami. W większości przypadków wystarcza ponowne uruchomienie skryptu, natomiast w starszych wersjach Petalinuxa bardzo często występują problemy związane z repozytoriami, dla których kod źródłowy został przeniesiony pod inny adres URL. W takim przypadku należy stworzyć dla danego pakietu łatkę z zaktualizowanym adresem w formacie „nazwa-pakietu_%.bbappend” i dołączyć do folderu projektu „/scieżka/do/Ultra96-PYNQ/Ultra96/petalinux_bsp_v2/meta-user/recipes-kernel”. Przygotowanie obrazu zajmuje od 2 do 4 godzin w zależności od wydajności komputera oraz łącza internetowego. Gotowy obraz oraz pliki uruchomieniowe, po wygenerowaniu należy wgrać na kartę MicroSD.

Karta dla podstawowych projektów i zastosowań według dokumentacji powinna posiadać rozmiar 8 GB, natomiast na potrzeby projektu wymagana jest z rozmiarem co najmniej 16 GB, ze względu na wykorzystanie w projekcie dodatkowych bibliotek i oprogramowania. Dodatkowo należy zwrócić uwagę na parametry karty, nie zostały one wymienione w dokumentacji, natomiast minimalnie powinna posiadać 10. klasę szybkości lub wyższą, aby zapewnić wysoką responsywność systemu Linux. Kartę przed wgraniem obrazu należy odpowiednio przygotować, wymagane jest stworzenie dwóch partycji. Przy pomocy programu „fdisk” systemu Linux zostały stworzone dwie partycje. Pierwsza partycja typu FAT o rozmiarze 512 MB dla plików rozruchowych „BOOT.BIN”, „boot.scr”, oraz „image.ub”. Natomiast reszta pamięci została

przydzielona do drugiej partycji, która została sformatowana jako typ EXT4, następnie na partycję zostały wypakowane pliki systemowe z archiwum „rootfs.tar.gz”.

4.3 Konfiguracja systemu Linux oraz instalacja wymaganych pakietów

4.3.1 Pierwsze uruchomienie układów

Przed pierwszym uruchomieniem płytki PYNQ-Z2 należało włożyć kartę MicroSD z przygotowanym wcześniej obrazem do odpowiedniego slotu i podłączyć kable USB Micro-B oraz Ethernet. Płytka posiada wbudowany moduł FTDI, który poprzez port USB pozwala na komunikację z komputerem. Układ udostępnia dwa porty, które wykorzystują protokoły szeregowe JTAG oraz UART. Podczas pierwszego uruchomienia należy upewnić się, że obraz wgrany na kartę uruchamia się poprawnie. W tym celu wykorzystany został port szeregowy UART, który pozwala na wyświetlenie w konsoli całego przebiegu uruchamiania układu. Na komputerze z systemem opartym o jądro Linux, układ przedstawia się jako konsola szeregowa pod nazwą „ttyUSB1”, natomiast w systemach Windows jako port szeregowy COM. Ze względu na pracę z wykorzystaniem komputera z systemem operacyjnym Linux, do komunikacji z układem został wykorzystany terminal „GTKTerm”, natomiast osoby pracujące na systemie Windows mogą wykorzystać do komunikacji program PuTTY. Wybrany program należało odpowiednio skonfigurować, poprzez ustawienie prędkości komunikacji na 115200 baud/s z 8-bitowym rozmiarem ramki, jednym bitem stopu oraz brakiem bitu parzystości. Natomiast na płytce należało ustawić odpowiednio zworkę odpowiedzialną za tryb uruchamiania na uruchomienie z kart SD oraz źródło zasilania na USB. Szczegółowa instrukcja oraz rozmieszczenie wyprowadzeń są dostępne na stronie producenta [4].

Płytka Ultra96-V2 nie posiada układu FTDI, natomiast posiada wyprowadzone złącza JTAG i UART o rozstawie 2 mm, które pozwalają na podłączenie

standardowych konwerterów UART i programatorów JTAG lub gotowego układu przygotowanego przez producenta płytki. Na płytce dostępne są również układ graficzny oraz wyjście Mini DisplayPort, które umożliwiają podłączenie monitora oraz wyświetlenie za jego pomocą przebiegu rozruchu układu i konsoli systemowej. W przypadku metody z wykorzystaniem monitora należy zwrócić uwagę na typ przewodów, układ wyświetli obraz tylko przy użyciu przewodu natywnego Mini DisplayPort na DisplayPort lub aktywnego adaptera. Przed uruchomieniem układu należało włożyć kartę z wygenerowanym obrazem systemu, podłączyć zasilacz, podłączyć kabel USB Micro-B oraz upewnić się, że mikro przełączniki zostały ustawione w pozycji odpowiedniej dla uruchamiania z karty SD. Dodatkowo, po prawnym uruchomieniu układ emuluje kartę sieciową oraz konsolę szeregową poprzez dostępny interfejs USB typu „device".

4.3.2 Konfiguracja dostępu do internetu

Przed przystąpieniem do instalacji potrzebnych pakietów należało zapewnić układom dostęp do internetu. W przypadku płytki PYNQ-Z2 układ należy podłączyć do urządzenia sieciowego przy użyciu przewodu Ethernet i poczekać aż zostanie przydzielony adres IP dla karty sieciowej lub może zostać manualnie przydzielony przy użyciu konsoli szeregowej. Płytka Ultra96-V2 nie posiada złącza RJ-45, natomiast została wyposażona w układ WiFi, który należy manualnie skonfigurować, aby uzyskać dostęp do internetu. Konfigurację można przeprowadzić na dwa sposoby, pierwszy sposób wymaga poprawnie uruchomionego systemu na płytce, który emuluje kartę sieciową. Po podłączeniu płytki do komputera system powinien wyświetlić komunikat o nowym urządzeniu sieciowym oraz przeprowadzić automatyczną konfigurację. Następnie należy uruchomić przeglądarkę oraz przejść do strony pod adresem „http://192.168.3.1", w przypadku wyświetlenia się komunikatu o autoryzacji należy wpisać hasło „xilinx". Po pomyślnej autoryzacji użytkownik zostanie przeniesiony do głównej strony środowiska, gdzie należy otworzyć widoczny katalog „common", uruchomić znajdujący się w nim skrypt „wifi.ipynb" w formie interaktywnej strony notatnika i postępować zgodnie z instrukcjami zawartymi w środku,

konfiguracja zostanie zachowana oraz zostanie uruchomiona przy kolejnym rozruchu systemu. Drugi sposób zakłada wykorzystanie konsoli szeregowej oraz pakietu „WPA Supplicant". W pierwszej kolejności należy wygenerować plik konfiguracyjny z nazwą oraz hasłem sieci, do której ma zostać podłączony układ, przy użyciu komendy: „`wpa_passphrase nazwa_sieci hasło_do_sieci > wpa.conf`", a następnie wykonać komendę: „`sudo wpa_supplicant -B -c ~/wpa.conf -i wlan0`", która podłączy układ do sieci bezprzewodowej. Natomiast w celu pozyskania adresu IP należy wykonać komendę: „`sudo dhclient wlan0`". Obie metody po poprawnej konfiguracji powinny zapewnić dostęp do internetu.

4.3.3 Instalacja oraz konfiguracja pakietów

Przed instalacją pakietów należało zaktualizować repozytoria i zaktualizować pakiety za pomocą komendy: „`sudo apt update && sudo apt upgrade -y`". W następnej kolejności należało zainstalować pakiet środowiska graficznego „XFCE4", który wykorzystuje mało zasobów podczas działania. Środowisko oraz potrzebne pakiety zostały zainstalowane za pomocą komendy: „`sudo apt install xfce4 xfce4-goodies tightvncserver autocutsel xfonts-base`". Następnie po zainstalowaniu środowiska należało zainstalować oraz skonfigurować serwer VNC, który pozwoli na interakcję za pomocą interfejsu graficznego z wykorzystaniem interfejsów sieciowych. Wybrany został pakiet „tightvncserver" do stworzenia serwera VNC, który pozwoli na dostęp do zdalnego pulpitu. Po zainstalowaniu pakietu należało uruchomić serwer komendą: „`vncserver`" w celu nadania hasła, a następnie wyłączyć serwer, aby dokończyć konfigurację. Rysunek 26 przedstawia podstawową konfigurację serwera, którą należy zapisać do pliku „~/.vnc/xstartup".

```
#!/bin/bash

xrdb $HOME/.Xresources
autocutsel -fork
startxfce4 &
```

Rysunek 26: Konfiguracja serwera VNC w pliku xstartup.

Ostatnim krokiem konfiguracji jest dołączenie kodu przedstawionego na Rysunku 27 do pliku „~/.bashrc", który uruchomi automatycznie serwer po włączeniu płytki.

```
SERVICE="Xtightvnc"
if pgrep -x "$SERVICE" >/dev/null
then
    echo "$SERVICE is running"
else
    rm -rf ~/.config/xfce4/panel
    rm -rf ~/.config/xfce4/xfconf/xfce-perchannel-xml/xfce4-panel.xml
    cd /home/xilinx/
    echo "$SERVICE starting"
    vncserver :1 -geometry 1600x900
fi
```

Rysunek 27: Kod odpowiedzialny za uruchamianie serwera VNC przy starcie systemu.

Zdalny pulpit dostępny jest pod adresem IP, który został przydzielony płytce, natomiast numer portu to 5901. Płytkę należało uruchomić ponownie, w celu zweryfikowania dostępu do zdalnego pulpitu.

W następnym kroku należało zainstalować biblioteki Python, które pozwalaj na stworzenie aplikacji okienkowej oraz wizualizację danych. W pierwszej kolejności należało zainstalować pakiety, od których zależą biblioteki Python. Pakiet „wxPython" wymaga bibliotek graficznych interfejsu GTK. Pakiety zostały pobrane za pomocą komendy przedstawionej na Rysunku 28.

```
xilinx@pynq:~$ sudo apt-get install dpkg-dev build-essential libjpeg-dev
libtiff-dev libsdl1.2-dev libgstreamer-plugins-base0.10-dev libnotify-dev
 freeglut3 freeglut3-dev libwebkitgtk-dev libghc-gtk3-dev libwxgtk3.0-gtk
3-dev
```

Rysunek 28: Komenda instalacyjna bibliotek wymaganych przez pakiet WX.

Natomiast instalacja bibliotek Python została przeprowadzona za pomocą komendy:
„`pip3 install wxpython==4.0.7 matplotlib numpy scipy`”.
Pakiet WX wymaga wspomagający modułów, które są dodatkowo kompilowane w trakcie instalacji biblioteki. W zależności od procesora, na którym jest instalowany pakiet, kompilacja może zająć od czterech do sześciu godzin.

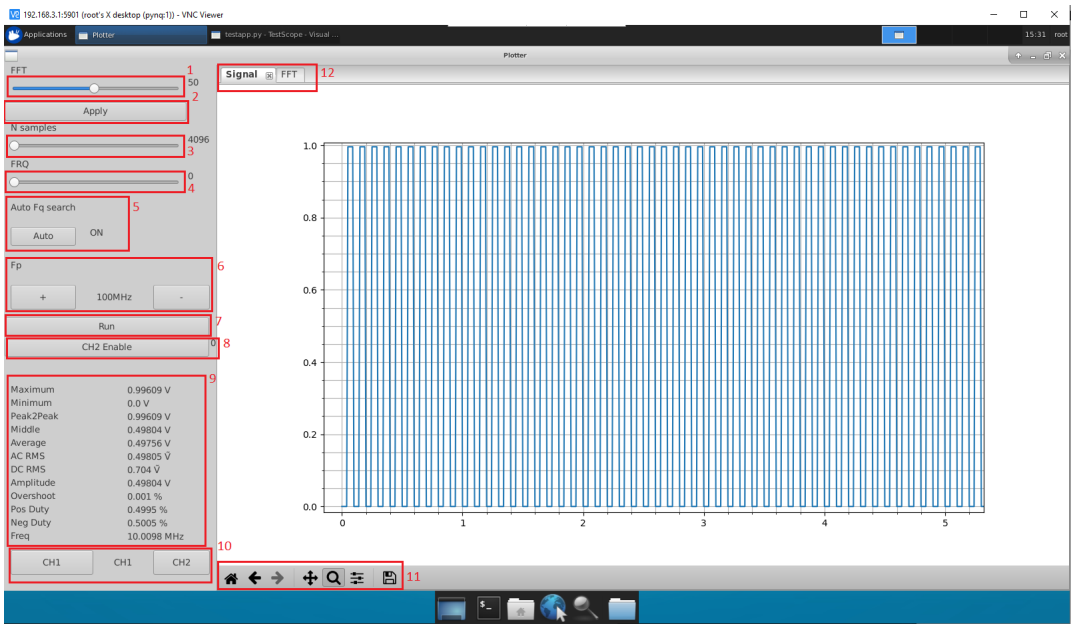
4.4 Aplikacja GUI

4.4.1 Interfejs aplikacji

Rozwój aplikacji w początkowych etapach prowadzony był na komputerze z architekturą x86_64. Praca na tradycyjnym komputerze PC pozwoliła na sprawną implementację części graficznej aplikacji oraz podstawowych funkcji do analizy sygnału, których funkcjonalność testowana była z wykorzystaniem plików zawierających dane sygnałów. Następnie rozwój został przeniesiony na płytki, na których aplikacja została zintegrowana ze środowiskiem PYNQ.

Interfejs został przedstawiony na Rysunku 29, jego wykonanie w projekcie zostało podzielone na dwie sekcje. Pierwsza sekcja zawiera elementy sterujące układem pomiarowym, który został przedstawiony w Rozdziale 3.3.1 oraz wyświetla podstawowe parametry mierzonego sygnału. Sekcja została zbudowana z wykorzystaniem widżetów dostępnych w środowisku wxWidgets, natomiast analiza sygnału została przeprowadzona z wykorzystaniem funkcji matematycznych dostępnych w bibliotekach „Numpy” oraz „SciPy”. Układ nie posiada fizycznego przetwornika analogowo-cyfrowego, dlatego próbki pomiarów generowane są przez bloki IP, które pozwalają zasymulować sygnały sinusoidalny oraz prostokątny. Bloki generujące zostały zaimplementowane wewnątrz logiki, a ich wyjścia sygnałowe zostały podłączone przez multiplekser do bloku IP z Rozdziału 3.3.1. Aplikacja okienkowa w pierwszej sekcji posiada również elementy sterujące zaimplementowanymi generatorami sygnałów, które pozwalają wybrać rodzaj i częstotliwość sygnału. Natomiast dla bloku IP odpowiedzialnego

za zmianę interfejsów został dodany suwak wyboru głębokości wyjściowego FIFO oraz przyciski zmiany częstotliwości sygnału odpowiadającego za gotowości danych na wejściu układu.



Rysunek 29: Interfejs graficzny aplikacji.

Druga sekcja odpowiedzialna jest za wyświetlenie mierzonego sygnału oraz jego reprezentacji w dziedzinie częstotliwości. Prezentacja danych została zaimplementowana z wykorzystaniem modułów pakietu „Matplotlib", które pozwalają wyświetlić oraz przesuwać i powiększać mierzone sygnały. Dodatkowo posiada moduł, który pozwala zapisywać dany sygnał w postaci obrazów. Dla okna, które odpowiada za przedstawienie widma sygnału, został dodany suwak pozwalający ograniczyć ilość wyświetlanych wartości częstotliwości na wykresie.

Aplikacja pracuje tylko w trybie pojedynczego pomiaru, który zostaje wymuszony przez użytkownika, co wynika z implementacji programu z wykorzystaniem tylko jednego wątku. Obecna implementacja aplikacji pozwala w sposób blokujący kolejno: odczytać, przetworzyć, a następnie wyświetlić sygnał mierzony na jednym kanale. Natomiast język Python pozwala również na wielowątkową oraz wieloprocesową pracę programów, która w kolejnej iteracji programu

umożliwiłaby zaimplementowanie ciągłego pomiaru zbliżonego do podglądu na żywo. Opóźnienie w podglądzie sygnału wynikałoby z tego, że kod napisany przy użyciu języka Python wykonywany jest przez interpreter, a nie kompilowany do języka maszynowego, jak jest to w przypadku języków C/C++.

4.4.2 Integracja aplikacji z bibliotekami PYNQ

Aplikacja wymaga zaimportowania biblioteki Python „pynq" do obsługi układów zaimplementowanych w logice. Biblioteka została zaimportowana w głównym kodzie programu poprzez użycie komendy: „import pynq". Biblioteka posiada wiele klas, które pozwalają na interakcję z układem logiki programowalnej oraz moduły do obsługi peryferiów takich, jak na przykład „AXI GPIO", wejścia oraz wyjścia audio i wideo, czy nakładki Arduino.

Środowisko wymaga najpierw zaprogramowania części logicznej układu, które realizowane jest poprzez inicjalizację obiektu typu „pynq.Overlay". Klasa „Overlay" podczas inicjalizacji przyjmuje ścieżkę do pliku bitstream jako argument. W trakcie inicjalizacji obiektu typu „Overlay" środowisko programuje układ plikiem podanym w argumencie, dodatkowo po pomyślnej inicjalizacji obiekt pozwala na wyświetlenie parametrów oraz informacji o architekturze zdefiniowanej w logice programowalnej, którą układ został zaprogramowany. Obiekt pozwala przeprogramować układ logiki, bez konieczności wykorzystania programatorów lub ponownego uruchamiania płytki. Użytkownik w dowolnym momencie może przeprogramować układ, natomiast pliki bitstream muszą zawierać skonfigurowany poprawnie układ procesorowy oraz adresy pamięci dla magistrali AXI. Kod odpowiedzialny za inicjalizację obiektu został przedstawiony poniżej: „ol = Overlay("/ścieżka/do/pliku/bitstream.bit")", natomiast informacje o blokach IP można uzyskać poprzez wykonanie kodu: „ol.ip_dict".

Klasa „MIMO" jest kolejnym elementem, który został wykorzystany do interakcji z układem logiki. Obiekt tej klasy pozwala na zapis i odczyt danych z rejestrów oraz adresowalnych bloków IP, które zostały przyporządkowane do adresów w

pamięci systemu. W projekcie aplikacji obiekt został wykorzystany do sterowania wyprowadzeniami GPIO, których wartości zmieniane są poprzez przyciski i suwaki w aplikacji okienkowej. Kod odpowiedzialny za interakcję z układem z wykorzystaniem obiektu został przedstawiony na Rysunku 30.

```
# Definicja adresów pamięci przypisanych do portów AXI GPIO
IP_BASE_ADDRESS = 0x80000000
ADDRESS_RANGE = 0xFFFF
ADDRESS_OFFSET = 0x08;
ADDRESS_OFFSET1 = 0x00;

# Stowrzenie obiektu
mmio = MMIO(IP_BASE_ADDRESS, ADDRESS_RANGE)
# Definicja metody odzczytu i zapisu na portch GPIO
def write_gpio(data):
  return mmio.write(ADDRESS_OFFSET1, data)

def read_gpio(addr):
  return mmio.read(addr)
```

Rysunek 30: Obiekt MMIO oraz funkcje zapisu i odczytu.

Funkcje zostają wywoływane podczas zdarzenia zmiany wartości lub stanu elementu interaktywnego interfejsu.

Układ odpowiedzialny za przekazywanie próbek do systemu procesorowego wymaga zaalokowania przestrzeni pamięci na dane. Dla aplikacji napisanych w języku C bufory danych alokowane są jako tablice, a następnie przekazywane do funkcji sterownika jako wskaźnik. Natomiast w środowisku PYNQ alokacja pamięci przeprowadzana jest przy użyciu klasy „Allocate” z biblioteki PYNQ. Obiekt zwracany po inicjalizacji jest typu „numpy.ndarray”, który posiada dodatkowe pola wymagane do interakcji z logiką. Przykład alokacji oraz wykorzystania bufora został przedstawiony na Rysunku 31.

```
def memory(N):
   nonlocal buffor
   buffor = pynq.allocate(shape=(N,), dtype=np.uint32)
   ol.fun_0.s_axi_mem_ptr.register_map.out_data_p_1 = buffor.physical_address
   ol.fun_0.s_axi_buf_size.register_map.buffsize = N
   ol.fun_0.s_axi_read_flag.register_map.rd = 0
```

Rysunek 31: Funkcja odpowiedzialna za inicjalizację pamięci próbek.

Rozdział V

Badania opracowanego oscyloskopu

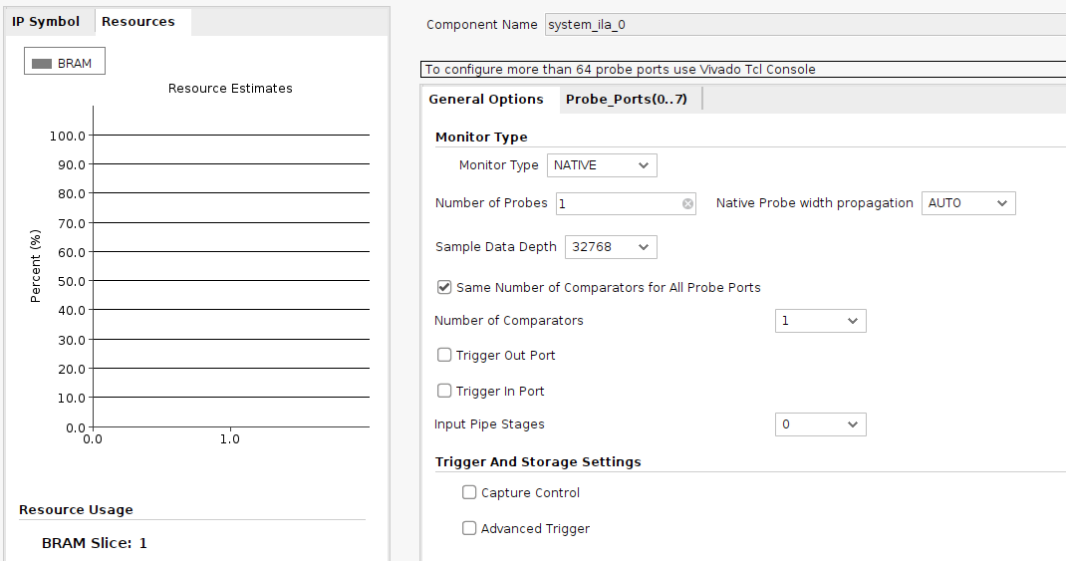
Przeprowadzone w ramach realizacji pracy badania zostały podzielone na dwie części. Pierwsza została poświęcona testom aplikacji oraz przedstawieniu wyników pomiarów. Natomiast w drugiej części została przetestowana wydajność układów z interfejsem AXI4-MM. Testy wydajności zostały wykonane przy użyciu notatników interaktywnych środowiska „Jupter Notebook" z interpreterem Python.

W przypadku wszystkich testów należało w pierwszej kolejności przesłać wygenerowane pliki „.bit" oraz „.hwh", które są potrzebne do zaprogramowania platformy. Pliki zostały przesłane do urządzeń z wykorzystaniem protokołu „SCP" (ang. Secure Copy Protocol). W celu przesyłu plików zapewniono dostęp urządzeń oraz komputera PC do to tej samej sieci lokalnej, a następnie zostały przesłane odpowiednie pliki dla każdej z platformy. Pliki zostały przesłane komendą: „`scp nazwa_pliku.bit xilinx@ip:/home/xilinx/nazwa_projektu/`". Płytki zostały podłączone do komputera PC poprzez interfejs JTAG, który pozwolił na podgląd sygnałów wewnątrz układu. W części sprzętowej do podglądu sygnału został wykorzystany wewnętrzny analizator stanów „ILA", natomiast za wizualizację danych odpowiedzialne było oprogramowanie Vivado.

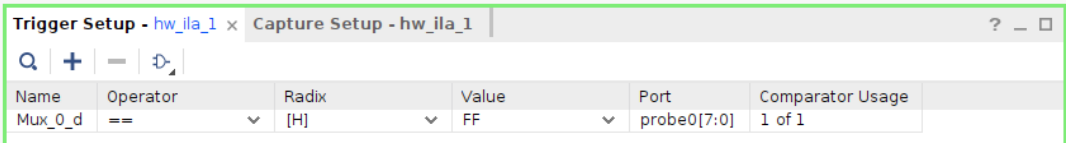
5.1 Test aplikacji GUI

W celu weryfikacji danych wygenerowanych przez układ oraz danych otrzymanych w aplikacji należało uruchomić program Vivado wraz z danym projektem oraz zweryfikować obecność konfiguracji sondy bloku IP „ILA". Do wewnętrznego analizatora stanów podłączone zostało wyjście multipleksera odpowiedzialnego za wybór generowanego sygnału, natomiast układ „ILA"

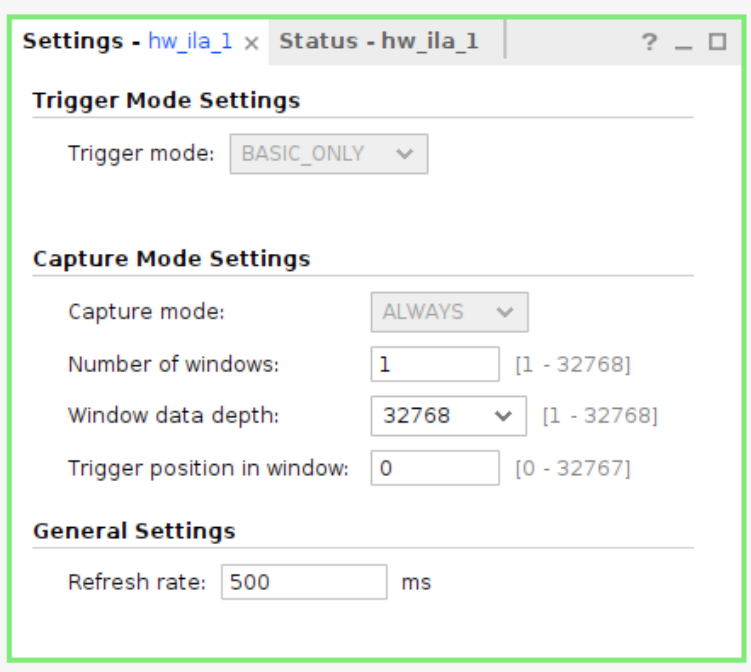
taktowany jest zegarem 200 MHz. Okno konfiguracyjne bloku IP analizatora „ILA” zostało przedstawione na Rysunku 32, natomiast ustawienia pomiarowe sondy zostały przedstawione na Rysunku 33 oraz Rysunku 34.



Rysunek 32: Ustawienia bloku IP „ILA”.



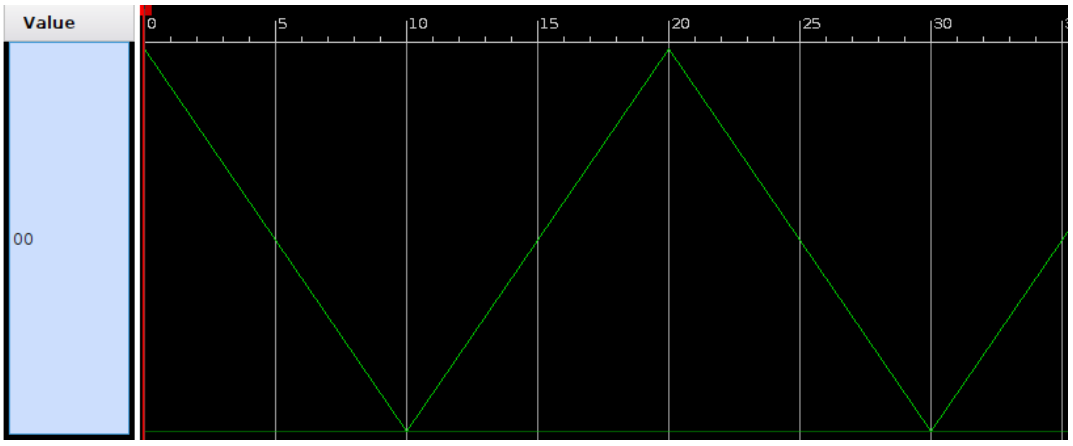
Rysunek 33: Ustawienie wyzwalacza sondy na wartość 0xFF.



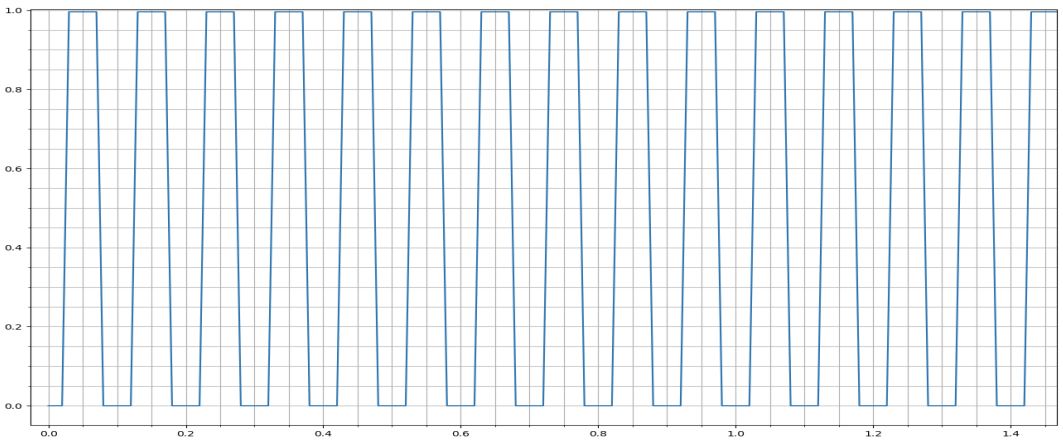
Rysunek 34: Ustawienie wielkości bufora sondy.

W kolejnym kroku należało uruchomić aplikację z uprawnieniami użytkownika „root”, poprzez wykonanie komendy: „`sudo python3 /ścieżka/do/pliku/app.py`”. Po uruchomieniu programu wykonane zostały cztery pomiary dla każdego z sygnałów, wyniki otrzymane na wykresie w aplikacji oraz przebieg sygnału odczytany przez sondę „ILA” zostały zapisane oraz przedstawione w formie rysunków.

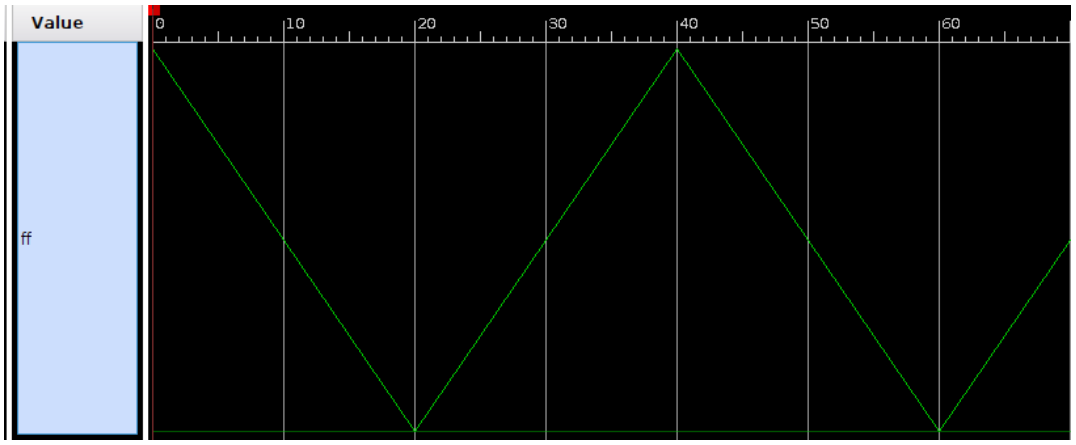
Pomiar sygnału prostokątnego o wypełnieniu 50% został wykonany dla częstotliwości: 10 MHz, 5 MHz, 1 MHz oraz 500 kHz. Analizator „ILA” posiada ograniczenie związane z wyświetlaniem sygnału prostokątnego. Układ przedstawia zmierzone sygnały w postaci sygnału trójkątnego, natomiast wartości w miejscach przedstawionych jako zbocza wynoszą odpowiednio „0” dla zboczy narastających oraz „255” dla zboczy opadających. Analizator nie podaje odstępu czasowego pomiędzy próbkami, dlatego częstotliwości sygnałów zostały policzone z wykorzystaniem częstotliwości próbkowania układu ILA.



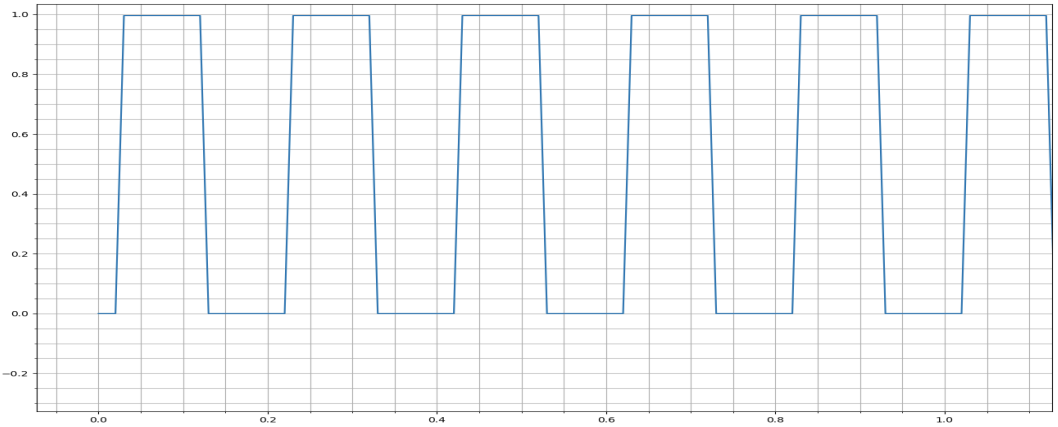
Rysunek 35: Sygnał 10 MHz zmierzony przez układ „ILA”.



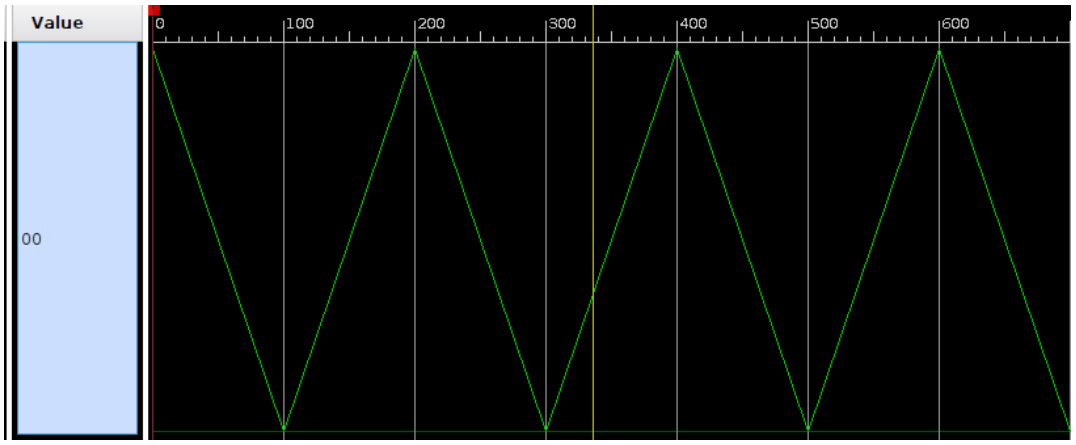
Rysunek 36: Sygnał 10 MHz zmierzony przez aplikację. Oś czasu w mikrosekundach.



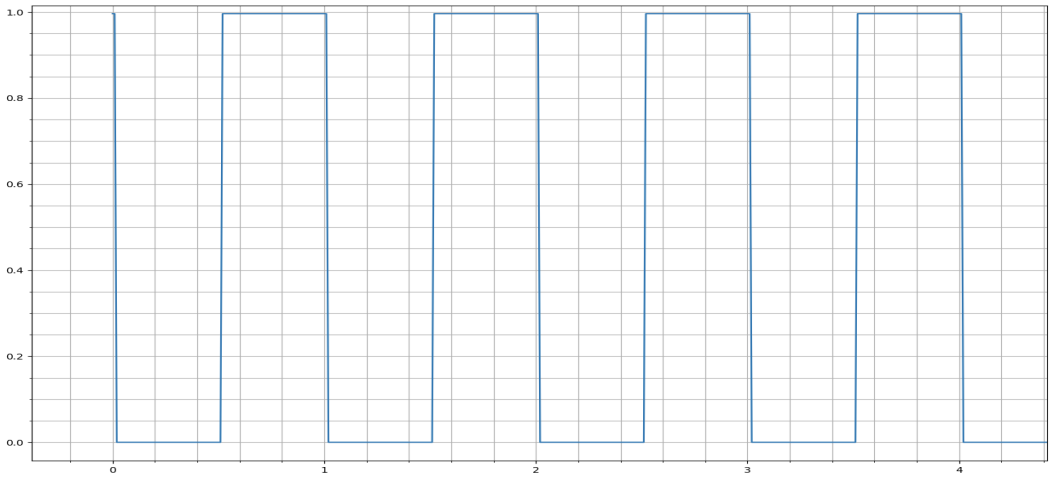
Rysunek 37: Sygnał 5 MHz zmierzony przez układ „ILA”.



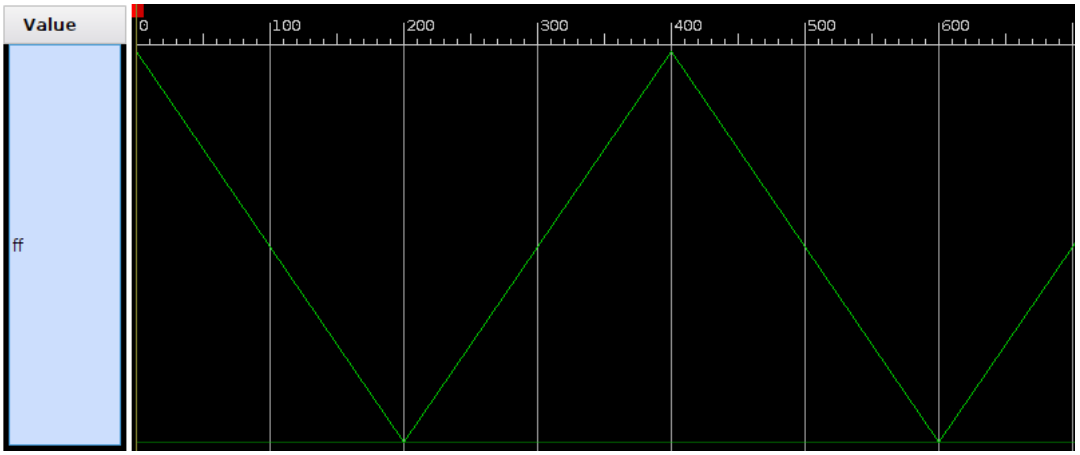
Rysunek 38: Sygnał 5 MHz zmierzony przez aplikację. Oś czasu w mikrosekundach.



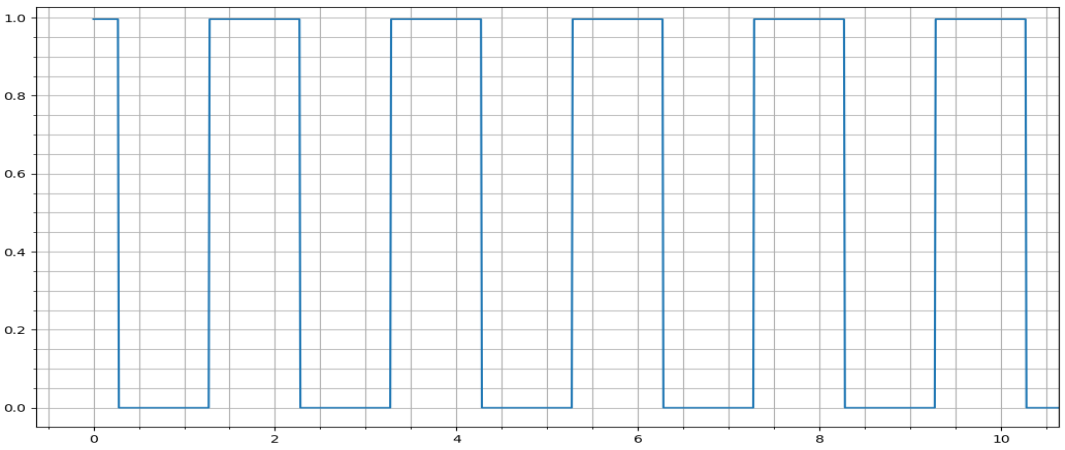
Rysunek 39: Sygnał 1 MHz zmierzony przez układ „ILA”.



Rysunek 40: Sygnał 1 MHz zmierzony przez aplikację. Oś czasu w mikrosekundach.



Rysunek 41: Sygnał 1 MHz zmierzony przez układ „ILA”.



Rysunek 42: Sygnał 500 kHz zmierzony przez aplikację. Oś czasu w mikrosekundach.

Przedstawione wykresy wskazują, że próbki sygnałów prostokątnych przekazywane są poprawnie do pamięci układu, natomiast aplikacja poprawnie odczytuje bufor oraz wyświetla sygnał. Dla każdego z sygnałów prostokątnych została wykonana analiza FFT oraz wyświetlona w postaci wykresu. Rysunek 43 przedstawia klasę odpowiedzialną za analizę FFT. Wyniki zostały przedstawione poniżej w formie Rysunków.

```python
class FFT_Func:
    def __init__(self, y, N, T, res):
        self.y = y
        self.N = N
        self.T = T
        self.res = res

    def FFT_Y(self):
        yf = np.fft.fft(self.y)
        return yf

    def FFT_X(self):
        xf = np.fft.fftfreq(self.N, self.T)[:self.N // 2]
        return xf

    def FFT_Freq(self):
        z = argrelextrema(2.0 / self.N * np.abs(self.FFT_Y()[0:self.N // 2]), np.greater, 0, self.res)
        z = np.concatenate(np.asarray(z))
        return z

    def FFT_Freq_Y(self):
        zy = np.ones(len(self.FFT_Freq())) * (2.0 / self.N * np.abs(self.FFT_Y()[self.FFT_Freq()]))
        # s = np.zeros(len(z))
        return zy
```

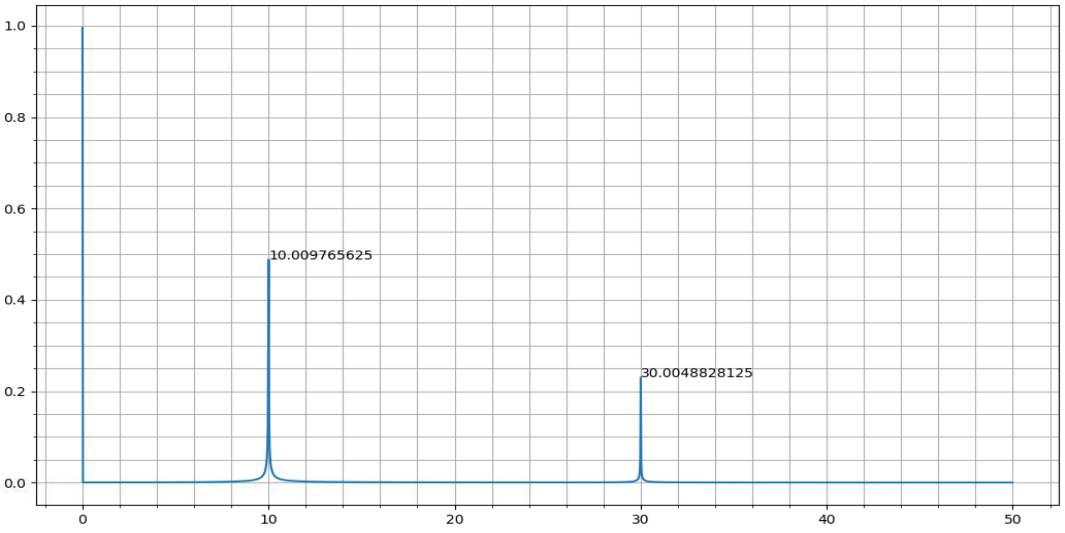
Rysunek 43: Klasa Python odpowiedzialna za analizę FFT sygnału.

Klasa „FFT_Func” podczas inicjalizacji obiektu przyjmuje jako parametry: bufor ze zmierzonym sygnałem, liczbę próbek, okres odpowiadający częstotliwości próbkowania oraz liczbę próbek okna, w którym szukane jest maksimum lokalne

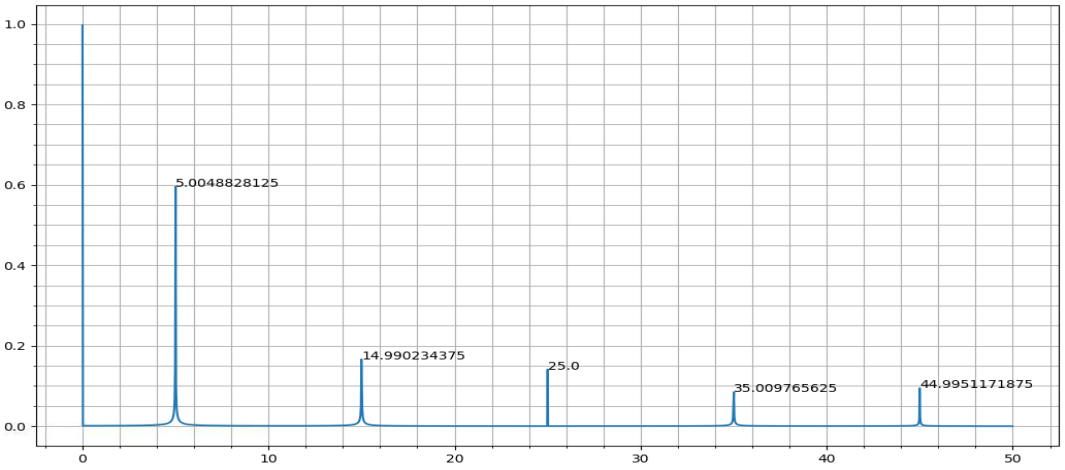
pośród obliczonych wartości częstotliwości. Następnie aplikacja wykorzystuje funkcje klasy do stworzenia wykresu oraz naniesienia wartości zgodnie z zadeklarowanym oknem. Rysunek 44 przedstawia kod odpowiedzialny za stworzenie wykresu FFT.

```
axe2 = plotter.add('FFT').gca()
for i, txt in enumerate(fft_ch.FFT_Freq() / (len(yx) * T)):
    axe2.annotate(txt, ((fft_ch.FFT_Freq()[i]) / (len(yx) * T), fft_ch.FFT_Freq_Y()[i]))
axe2.plot(fft_ch.FFT_X(), 2.0 / len(yx) * np.abs(fft_ch.FFT_Y()[0:len(yx) // 2]))
axe2.minorticks_on()
axe2.grid(True, which='both', axis='both')
```

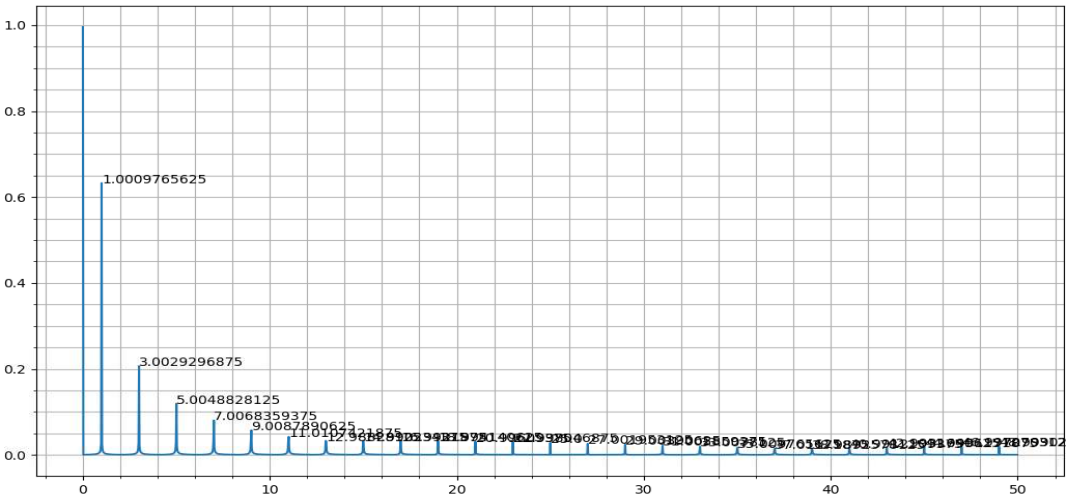
Rysunek 44: Generacja zakładki z wykresem FFT.



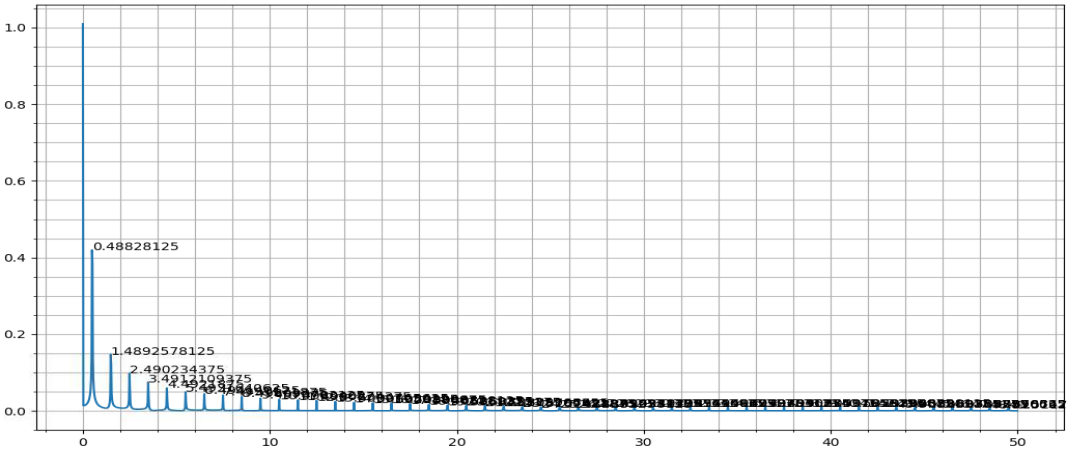
Rysunek 45: Analiza FFT sygnału 10 MHz. Oś czasu w megahercach.



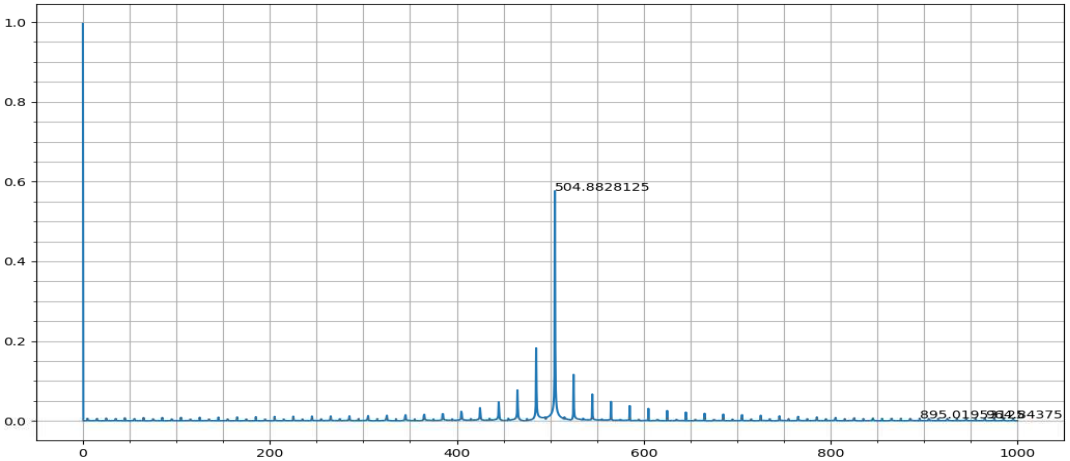
Rysunek 46: Analiza FFT sygnału 5 MHz. Oś czasu w megahercach.



Rysunek 47: Analiza FFT sygnału 1 MHz. Oś czasu w megahercach.



Rysunek 48: Analiza FFT sygnału 500 kHz. Częstotliwość próbkowania 100 MHz. Oś czasu w kilohercach.



Rysunek 49: Analiza FFT sygnału 500 kHz. Częstotliwość próbkowania 2 MHz. Oś czasu w kilohercach.

Analiza FFT sygnałów została przeprowadzona na sygnałach zmierzonych z częstotliwością 100 MHz, natomiast dla sygnału o częstotliwości 500 kHz został wykonany dodatkowy pomiar z częstotliwością 2 MHz. Większa częstotliwość pozwoliła na wyświetlenie większej ilości sygnałów składowych, natomiast mniejsza częstotliwość próbkowania pozwoliła na dokładniejsze zmierzenie częstotliwości sygnału.

Aplikacji pozwala również wyświetlić podstawowe parametry sygnału, takie jak:

- wartość maksymalną,
- wartość minimalną,
- wartość „peak-to-peak”,
- wartość połowy sygnału,
- wartość średnią,
- wartość AC RMS,
- wartość DC RMS,
- wartość amplitudy,
- procent przeregulowania,
- współczynnik wypełnienia sygnału dla połowy dodatniej,
- współczynnik wypełnienia sygnału dla połowy ujemnej,
- częstotliwość sygnału.

Całość została zaimplementowana w postaci klasy Python, natomiast kod został przedstawiony na Rysunku 50, natomiast Rysunek 51 przedstawia wyniki dla sygnału o częstotliwości 10 MHz.

```
class Meas:
    def __init__(self, sig):
        self.sig = sig

    def ValMax(self):
        v_max = np.max(self.sig)
        return round(v_max, 5)

    def ValMin(self):
        v_min = np.min(self.sig)
        return round(v_min, 5)

    def ValPeak(self):
        v_peak = abs(self.ValMin()) + abs(self.ValMax())
        return round(v_peak, 5)

    def ValMid(self):
        v_mid = (self.ValMin() + self.ValMax()) / 2
        return round(v_mid, 5)

    def ValAvg(self):
        v_avg = np.average(self.sig)
        return round(v_avg, 5)

    def ValACRMS(self):
        v = 0
        for i in range(len(self.sig)):
            v = v + (self.sig[i] - self.ValAvg()) ** 2
        v = v / len(self.sig)
        v_RMS = np.sqrt(v)
        return round(v_RMS, 5)

    def ValDCRMS(self):
        v = 0
        for i in range(len(self.sig)):
            v = v + (self.sig[i] ** 2)
        v = v / len(self.sig)
        v_RMS = np.sqrt(v)
        return round(v_RMS, 5)

    def ValAmp(self):
        v_amp = (self.ValMax() - self.ValMin()) / 2
        return round(v_amp, 5)

    def ValOver(self):
        v = self.ValAmp()
        if math.isnan(v) or v == 0:
            return 0
        else:
            return round(100 * (self.ValPeak() / v / 2 - 1), 4)

    def PosDuty(self):
        v_sum = sum(i > self.ValMid() for i in self.sig) / len(self.sig)
        return round(v_sum, 4)

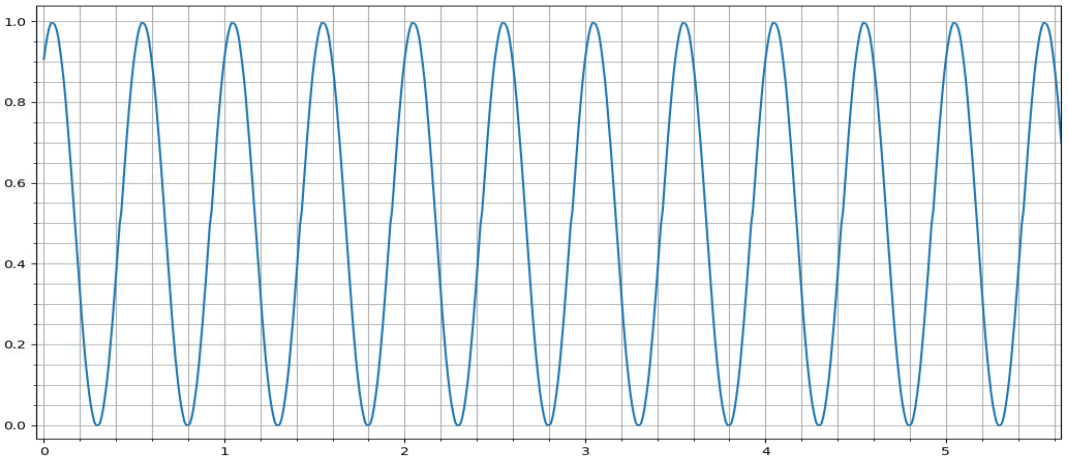
    def NegDuty(self):
        v_sum = 1 - self.PosDuty()
        return round(v_sum, 4)
```

Rysunek 50: Klasa odpowiedzialna za obliczanie parametrów sygnału.

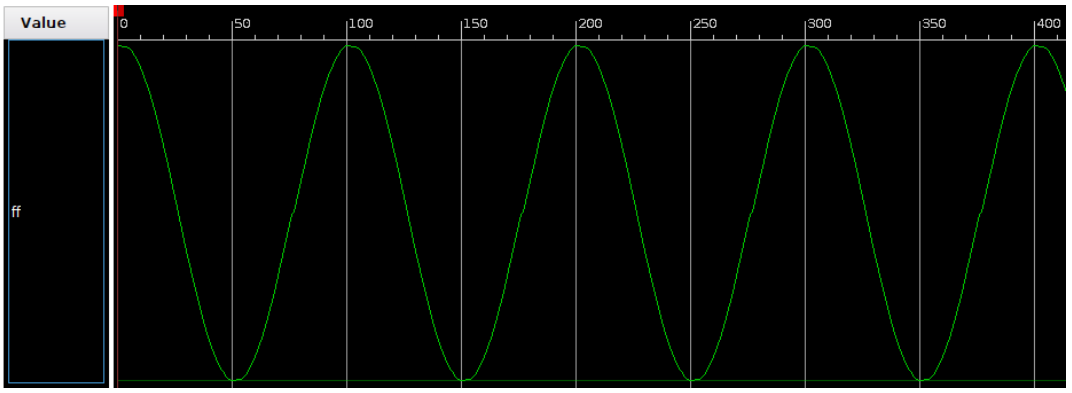
Maximum	0.99609 V
Minimum	0.0 V
Peak2Peak	0.99609 V
Middle	0.49804 V
Average	0.49805 V
Amplitude	0.49804 V
Overshoot	0.001 %
Pos Duty	0.5 %
Neg Duty	0.5 %
Freq	10.0098 MHz

Rysunek 51: Parametry sygnału prostokątnego o częstotliwości 10 MHz.

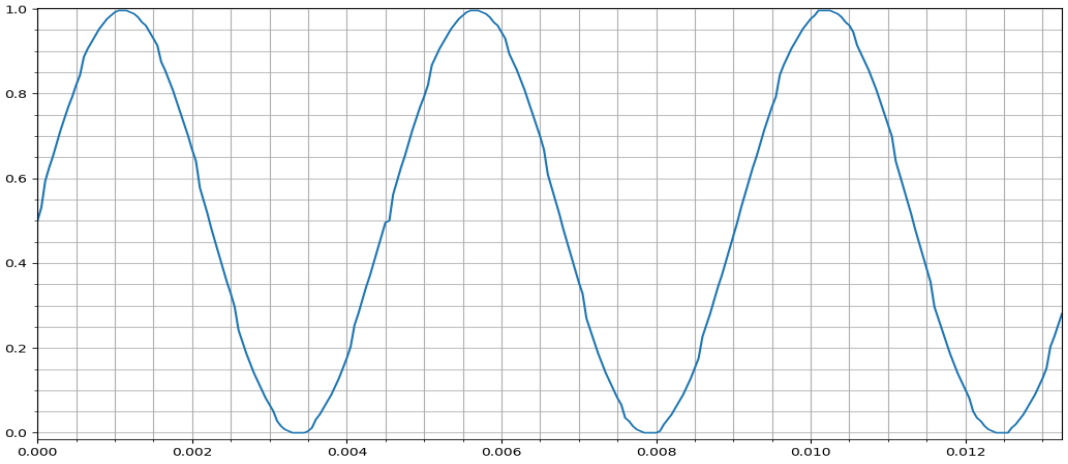
Następnie aplikacja została przetestowana z wykorzystaniem sygnału sinusoidalnego o częstotliwościach 2 MHz, 200 kHz, 100 kHz i 20 kHz. Wykresy oraz parametry sygnałów zostały przedstawione na Rysunkach 52 do 59.



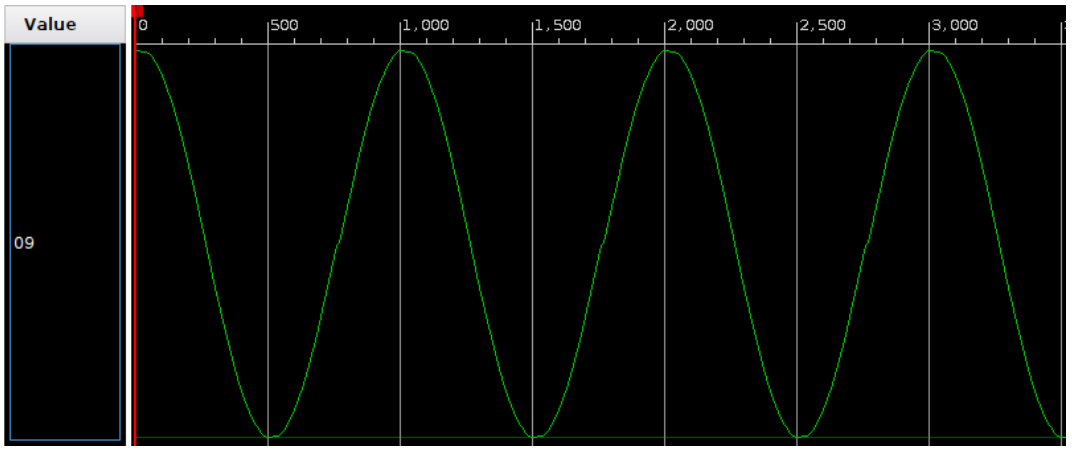
Rysunek 52: Sygnał 2 MHz zmierzony przez aplikację. Oś czasu w mikrosekundach.



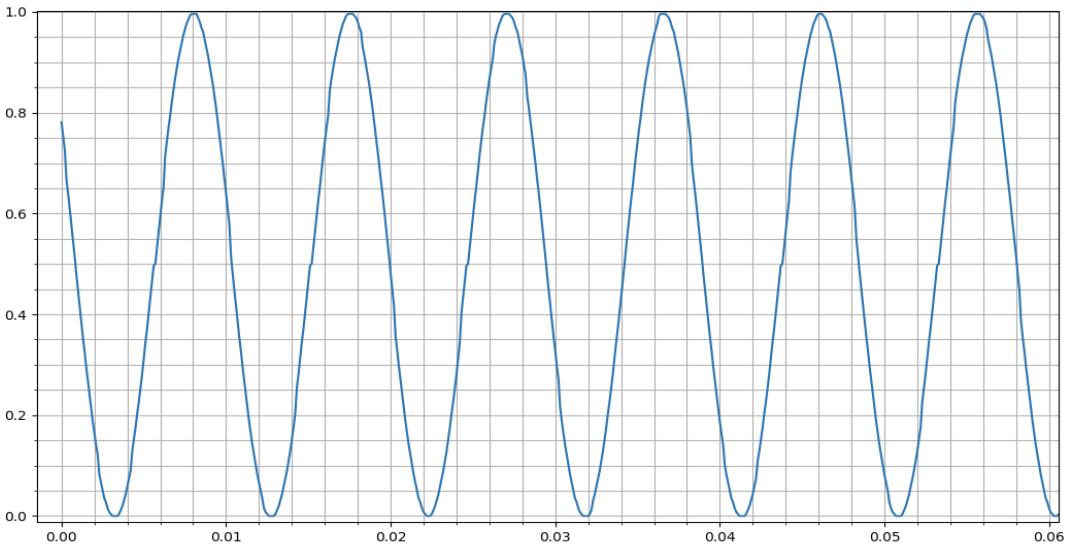
Rysunek 53: Sygnał 2 MHz zmierzony przez układ „ILA”.



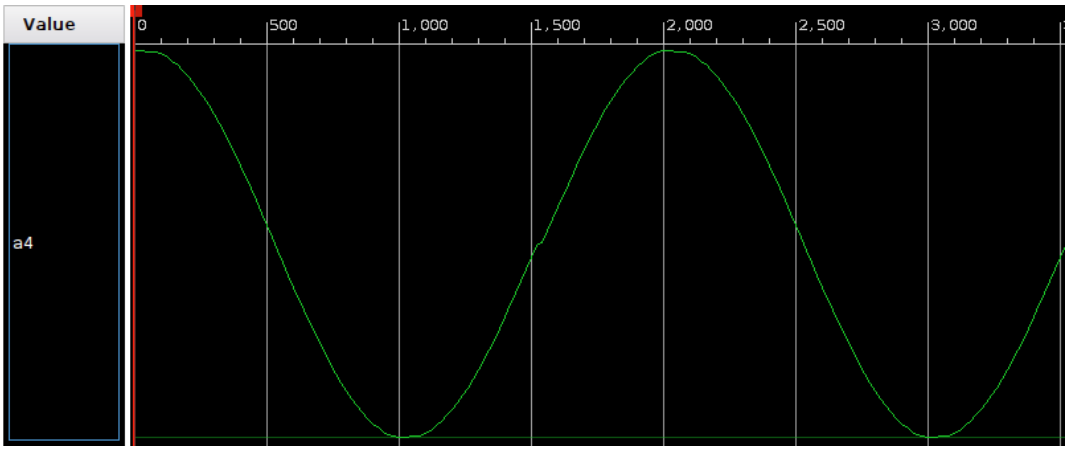
Rysunek 54: Sygnał 200 kHz zmierzony przez aplikację. Oś czasu w milisekundach.



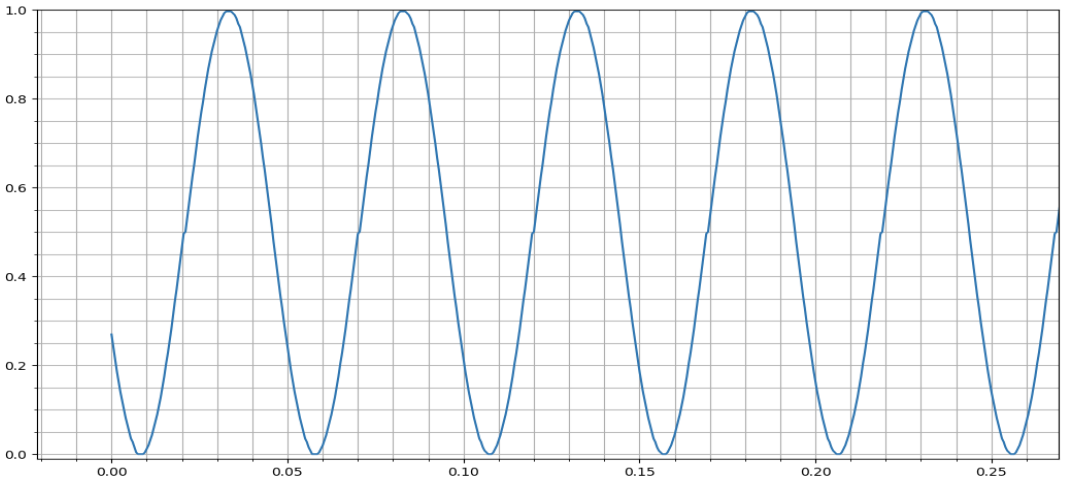
Rysunek 55: Sygnał 200 kHz zmierzony przez układ „ILA”.



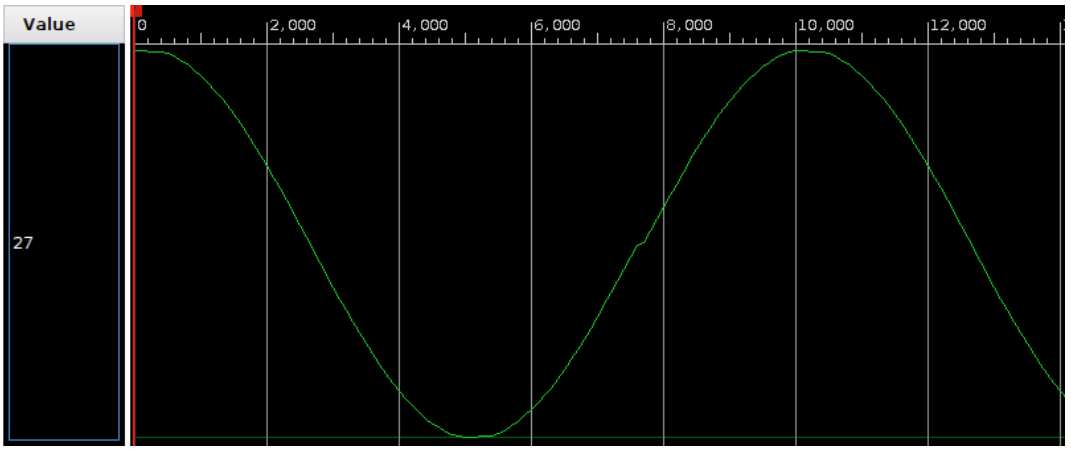
Rysunek 56: Sygnał 100 kHz zmierzony przez aplikację. Oś czasu w milisekundach.



Rysunek 57: Sygnał 100 kHz zmierzony przez układ „ILA”.



Rysunek 58: Sygnał 20 kHz zmierzony przez aplikację. Oś czasu w milisekundach.



Rysunek 59: Sygnał 100 kHz zmierzony przez układ „ILA”.

Test aplikacji wykazał, że sygnały generowane w logice programowalnej są poprawnie odczytywane przez układ procesorowy oraz środowisko PYNQ. Sygnały są poprawnie prezentowane na wykresie, natomiast obliczone przez aplikację posiadały niewielki, nieprzekraczający 0,5% błąd pomiaru. Wyniki pomiaru częstotliwości przedstawione zostały w Tabeli 1 oraz Tabeli 2.

Częstotliwość generowana	Częstotliwość obliczona	Błąd względny pomiaru
10 MHz	10,0098 MHz	0,098%
5 MHz	5,0049 MHz	0,098%
1 MHz	1.001 MHz	0,1%
500 kHz	504,8828 kHz	0,098%

Tabela 1: Wyniki pomiaru częstotliwości dla sygnału prostokątnego

Częstotliwość generowana	Częstotliwość obliczona	Błąd względny pomiaru
2 MHz	2,002 MHz	0,1%
200 kHz	200,9277 kHz	0,46%
100 kHz	101,0742 kHz	0,1%
20 kHz	20,0195 kHz	0,098%

Tabela 2: Wyniki pomiaru częstotliwości dla sygnału sinusoidalnego

5.2 Badanie wydajności opracowanego systemu

Testy wydajności zostały przeprowadzone przy użyciu notatników Jupyter. Układ testowy wykorzystuje blok IP „AXI Traffic Generator” do wygenerowania ruchu na interfejsie AXI-MM w trzech trybach pracy: tylko zapis, tylko odczyt oraz zapis i odczyt jednocześnie. Natomiast blok IP „AXI Performance Monitor” został wykorzystany do pomiarów wygenerowanego ruchu. Czas każdego z pomiarów wynosi 1 ms. Dodatkowo dla układu Ultra96-V2 zostały przygotowane dwa zestawy pomiarowe. W pierwszym zostały ustawione maksymalne wartości częstotliwość interfejsu AXI oraz szerokość szyny danych. Natomiast w drugim zestawie wartości częstotliwości i szerokość szyny zostały ustawione takie same jak w układzie PYNQ-Z2, co pozwoliło porównać wydajność pomiędzy architekturami. Kod programu wykorzystany do przeprowadzenia pomiarów został przedstawiony na rysunkach poniżej.

Pomiar wydajności układów

```
import pynq
import numpy as np
from time import sleep
```

Inicjalizacja obiektu Overlay

```
ol = pynq.Overlay("traffic_gen_wr.bit")
```

Lista dotępnych układów

```
print(ol.ip_dict.keys())
```

```
dict_keys(['axi_gpio_0', 'axi_perf_mon_0', 'axi_traffic_gen_0', 'zynq_ultra_ps_e_0'])
```

```
GPIO_IP_BASE_ADDRESS = 0xA0000000
GPIO_ADDRESS_RANGE = 0xFFFF
GPIO_ADDRESS_OFFSET = 0x00;
```

```
mmio_gpio = pynq.MMIO(GPIO_IP_BASE_ADDRESS, GPIO_ADDRESS_RANGE)
```

Spis adresów dla rejestrów bloku AXI Performance Monitor

```
PERF_IP_BASE_ADDRESS = 0xA0010000
PERF_ADDRESS_RANGE = 0xFFFF
PERF_ADDRESS_OFFSET = 0x00
PERF_METRIC_SICR_0 = 0x0028
PERF_METRIC_SEL_0 = 0x0044
PERF_METRIC_CNT_0 = 0x0100
PERF_METRIC_RAN_0 = 0x0108
PERF_METRIC_EN_0 = 0x010C
PERF_METRIC_CTR_0 = 0x0300
```

```
mmio_perf = pynq.MMIO(PERF_IP_BASE_ADDRESS, PERF_ADDRESS_RANGE)
```

Rysunek 60: Inicjalizacja obiektów „Overlay” oraz „MMIO”.

Obiekt „mmio_gpio” posłużył do uruchamiania układu generującego sygnał rozpoczęcia i zakończenia pomiaru. Natomiast obiekt „mmio_perf” został wykorzystany do konfiguracji układu pomiarowego „AXI Performance Monitor”.

```
# Całkowita ilość traskacji zapisu
WRITE_TRANS_COUNT = 0x00
# Całkowita ilość traskacji odczytu
READ_TRANS_COUNT = 0x01
# Całkowita ilość przesłanych bajtów podczas zapisu
WRITE_BYTE_COUNT = 0x02
# Całkowita ilość przesłanych bajtów podczas odczytu
READ_BYTE_COUNT = 0x03
# Całkowity czas pojedyńczego transferu odczytu liczony
# od zlecenia odczytu do zakończenia odczytu ostanigo bajtu
AVG_READ_LATENCY = 0x05
# Całkowity czas pojedyńczego transferu zapisu liczony
# od zlecenia zapisu do zakończenia zapisu ostanigo bajtu
AVG_WRITE_LATENCY = 0x06
# Całkowita ilość pustych cykli powodowanych przez układ Slave
# podczas zapisu. Wartość odpowiada ilości cykłów zegarowych
# pomiędzy ustawienie sygnałów WVALID i WREADY.
SLV_WR_IDLE_COUNT = 0x07
# Całkowita ilość pustych cykli powodowanych przez układ Master
# podczas odczytu. Wartość odpowiada ilości cykłów zegarowych
# pomiędzy ustawienie sygnałów RVALID i RREADY.
MST_RD_IDLE_COUNT = 0x08
```

Rysunek 61: Spis ustawień pomiarowych w układzie „AXI Performance Monitor”.

Rysunek 61 przedstawia spis wartości dla rejestru odpowiedzialnego wybór pomiaru w układzie „AXI Performance Monitor”, które zostały wykorzystane do przetestowania obu platform. Natomiast na Rysunku 62 zostały przedstawione funkcje odpowiedzialne za rekonfigurację układu pomiarowego oraz rozpoczęcie i zakończenie pomiaru. Na Rysunku 63 został przedstawiony kod odpowiedzialny za wykonanie pomiaru liczby bajtów przesłanych podczas transakcji zapisu.

Funkcja resetująca układ pomiarowy

```
def SetAxiPerfMon():
    VAL = 0x10 << 16 | 0x08
    mmio_perf.write(PERF_METRIC_RAN_0,VAL)

    mask = mmio_perf.read(PERF_METRIC_CTR_0)
    mask |= 0x01
    mmio_perf.write(PERF_METRIC_CTR_0,mask)

    mask = mmio_perf.read(PERF_METRIC_SICR_0)
    mask |= (0x01 << 8)
    mmio_perf.write(PERF_METRIC_SICR_0,mask)
```

Funkcja rozpoczynająca pomiar

```
def StartMeas():
    mmio_gpio.write(GPIO_ADDRESS_OFFSET,0x00)
    sleep(0.1)
    mmio_gpio.write(GPIO_ADDRESS_OFFSET,0x01)
    sleep(0.00001)
    mmio_gpio.write(GPIO_ADDRESS_OFFSET,0x03)
    sleep(0.00001)
    mmio_gpio.write(GPIO_ADDRESS_OFFSET,0x01)
```

Rysunek 62: Funkcja rekonfiguracji układu pomiarowego oraz funkcja odpowiedzialna za rozpoczęcie pomiaru.

1. Ustawienie pomiaru na zliczanie przesłanych bajtów (WRITE_BYTE_COUNT)

```
mmio_perf.write(PERF_METRIC_SEL_0,WRITE_BYTE_COUNT)
SetAxiPerfMon()
```

Wykonanie pomiaru i odczyt wartośći licznika

```
StartMeas()
current_val = mmio_perf.read(PERF_METRIC_CNT_0)
write_byte_count = current_val
z[0] = write_byte_count
write_byte_count
```

Rysunek 63: Konfiguracja układu pomiarowego oraz wykonanie pomiaru.

	Ultra96-V2 wartości maksymalne	Ultra96-V2	PYNQ-Z2
Liczba bajtów oczytanych	1959936	1058816	1024000
Liczba transakcji odczytu	954	507	500
Całkowity czas trwania transakcji odczytu	781013	398426	396600
Liczba pustych cykli generowanych przez układ „Master”	0	0	0

Tabela 3: Wyniki pomiarów trybu tylko odczytu.

Pomiary zostały powtórzone pięciokrotnie, następnie wyniki zostały uśrednione oraz wpisane do tabel pomiarowych. Wyniki otrzymane podczas testu układu generującego tylko transakcje odczytu zostały przedstawione w Tabeli 3. Architektura Zynq UltraScale+ nie wykazuje dużo wyższej wydajności względem architektury Zynq-7000, dla testów z tymi samymi prędkościami szyny danych oraz częstotliwością zegara interfejsu AXI. Zwiększenie zegara oraz szyny danych pozwoliło uzyskać przyrost wydajności na poziomie 193% względem architektury Zynq-7000.

	Ultra96-V2 wartości maksymalne	Ultra96-V2	PYNQ-Z2
Liczba bajtów zapisanych	2658304	1308672	1599488
Liczba transakcji zapisu	1298	545	781
Całkowity czas trwania transakcji zapisu	917388	399572	578745
Liczba pustych cykli generowanych przez układ „Slave”	1	0	2

Tabela 4: Wyniki pomiarów trybu tylko zapisu.

Wyniki testu dla układu generującego tylko transakcje zapisu zostały przedstawione w Tabeli 4. Dla tego trybu pomiaru układ z architekturą Zynq-7000 przesłał większą liczbę bajtów, względem układu z architekturą Zynq UltraScale+. Natomiast w przypadku układu z wyższymi parametrami zysk wydajności dla operacji zapisu wynosi 164%. W obu układach wystąpiło również oczekiwanie na proces „handshake", który w przypadku płytki PYNQ-Z2 wyniósł dwa cykle zegarowe, natomiast dla płytki Ultra96-V2 z wyższymi parametrami był to jeden cykl zegara.

	Ultra96-V2 wartości maksymalne	Ultra96-V2	PYNQ-Z2
Liczba bajtów zapisanych	2588672	1126400	765952
Liczba bajtów oczytanych	247808	1540096	753664
Liczba transakcji zapisu	1281	626	373
Liczba transakcji odczytu	107	754	363
Całkowity czas trwania transakcji odczytu	607431	599959	397950
Całkowity czas trwania transakcji zapisu	149670	470179	478968
Liczba pustych cykli generowanych przez układ „Slave"	0	49984	105027
Liczba pustych cykli generowanych przez układ „Master"	0	0	0

Tabela 5: Wyniki pomiarów trybu jednoczesnego odczytu i zapisu.

Tabela 5 przedstawia wyniki dla testu transakcji zapisu i odczytu jednocześnie. W przypadku tego testu architektura Zynq UltraScale+ wykazuje się przyrostem wydajności na poziomie 142%. Dodatkowo w żadnym z układów nie wystąpiły puste cykle w oczekiwaniu na układ „Master". Natomiast dla płytki Ultra96-V2 z

niższymi parametrami interfejsu oraz płytki PYNQ-Z2 wystąpiły puste cykle w oczekiwaniu na gotowość układu „Slave” podczas transmisji zapisu.

Wyniki badań wskazują, że magistrala AXI w obu układach umożliwia implementację bloków IP o wysokiej przepustowości danych. W implementacji oscyloskopu cyfrowego z szybkimi układami przetworników analogowo-cyfrowych wysoka przepustowość jest niezbędna. Wartość przepustowości uzyskana w teście jednoczesnego zapisu i odczytu dla płytki PYNQ-Z2 pozwoliłaby na zaimplementowanie bloku IP dla układów z protokołem JESD204 w standardzie JESD204A. Natomiast w implementacji oscyloskopu cyfrowego, który wykorzystuje tylko transakcje odczytu, układ umożliwia implementację bloku IP interfejsu JESD204 w standardzie JESD204B. Wydajność płytki Ultra96-V2 umożliwia zaimplementowanie bloku IP z protokołem JESD204 w standardzie JESD204B oraz JESD204C.

Rozdział VI

Podsumowanie i wnioski

Celem projektu było stworzenie oscyloskopu cyfrowego w układzie SoC Zynq z wykorzystaniem środowiska PYNQ. W ramach projektu zostały wykonane między innymi: obrazy systemów dla płyt rozwojowych, blok IP zamieniający interfejs równoległy na interfejs AXI, aplikacja GUI oraz oprogramowanie do badania wydajności interfejsów AXI. Dla każdego układu zostały przygotowane odpowiednie projekty w oprogramowaniu Vivado oraz programy testujące w formie interaktywnych notatników środowiska „Jupyter Notebook".

Blok IP, który został wykorzystany do zamiany interfejsu równoległego na interfejs AXI, został napisany w języku C oraz wygenerowany w oprogramowaniu Vitis HLS. Następnie został zintegrowany w schemacie blokowym architektury, w którym został podłączony do układu procesorowego Zynq. Pliki potrzebne do zaprogramowania układów zostały wygenerowane przy użyciu oprogramowania Vivado. W projekcie została wykorzystana płytka PYNQ-Z2 firmy TUL, która wyposażona została w układ SoC Zynq-7000 oraz płytka Ultra96-V2 firmy Avnet z układem Zynq UltraScale+.

Interfejs aplikacji GUI został stworzony z wykorzystaniem biblioteki wxPython, natomiast wizualizacja mierzonego sygnału została wykonana przy użyciu biblioteki Matplotlib. Za obliczanie parametrów sygnałów odpowiedzialne były funkcje dostępne w bibliotekach Numpy i SciPy. Integracja aplikacji GUI ze środowiskiem PYNQ pozwoliła na komunikację oraz zarządzanie strukturą logiki programowalnej. Po wykonaniu aplikacji należało zweryfikować działanie bloku IP odpowiedzialnego za zamianę interfejsu oraz poprawność wyświetlanych danych w aplikacji. Do weryfikacji sygnałów odczytywanych przez aplikację

posłużył blok IP „ILA”, który pozwolił na podgląd mierzonego sygnału w oprogramowaniu Vivado.

Badania zrealizowane w ramach pracy obejmowały porównanie wydajności platform Zynq-7000 i Zynq UltraScale+, przedstawienie oraz analizę wyników, pod względem możliwości układów w zastosowaniu jako oscyloskop cyfrowy. Wyniki wykazały możliwość implementacji szybkich interfejsów powszechnie wykorzystywanych w układach przetworników analogowo-cyfrowych. Układy Zynq zastosowane na płytkach pomimo najniższej klasy prędkości umożliwiają stworzenie szybkiego oscyloskopu cyfrowego. Natomiast wykorzystanie środowiska PYNQ oraz interaktywnych notatników pozwoliło na sprawne i czytelne przeprowadzenie badań.

Obliczenia dla analizy FFT zostały wykonane z wykorzystaniem biblioteki SciPy, która cechuje się niską wydajnością ze względu na wykonywanie obliczeń w systemie procesorowym. W ostatecznej implementacji oscyloskopu obliczenia dla analizy FFT powinny zostać zrealizowane w układzie logiki programowalnej z wykorzystaniem akceleratorów. Aplikacja powinna umożliwiać pracę wielowątkową, która dodatkowo zwiększy wydajność w obliczeniach na procesorze.

Podsumowując wykonane prace i przeprowadzone badania należy uznać, że cel pracy został osiągnięty, a szczegółowy zakres zrealizowany.

Literatura

[1] Tektronix, Oscilloscope Types, dostęp z dnia 20.10.2022, https://www.tek.com/document/online/primer/xyzs-scopes/ch2/oscilloscope-types

[2] ST, Datasheet STM32F446xC/E, 2021, https://www.st.com/resource/en/datasheet/stm32f446re.pdf

[3] Xilinx Inc., Zynq-7000 Block Diagram, dostęp z dnia 20.10.2022, https://www.xilinx.com/content/dam/xilinx/imgs/block-diagrams/zynq-mp-core-dual.png

[4] TUL., PYNQ-Z2 Reference Manual v1.0, 2018 https://dpoauwgwqsy2x.cloudfront.net/Download/pynqz2_user_manual_v1_0.pdf

[5] Avnet, Ultra96-V2 Getting Started Guide 2.0, 2020, https://www.avnet.com/wps/wcm/connect/onesite/f21462c6-4997-41a2-a95e-80122b73aea9/Ultra96-V2-GSG-v2_0.pdf?MOD=AJPERES&CACHEID=ROOTWORKSPACE.Z18_NA5A1I41L0ICD0ABNDMDDG0000-f21462c6-4997-41a2-a95e-80122b73aea9-obVX12W

[6] ARM, AMBA AXI and ACE Protocol Specification Version H.c, 2021, https://developer.arm.com/documentation/ihi0022/hc/?lang=en

[7] Xilinx Inc, Vivado Design Suite: AXI Reference Guide (UG1037), 2017 https://docs.xilinx.com/v/u/en-US/ug1037-vivado-axi-reference-guide

[8] Texas Instruments, JESD204B Overview, 2016, https://www.ti.com/lit/ml/slap161/slap161.pdf

[9] Xilinx Inc., Vitis High-Level Synthesis User Guide (UG1399), 2021, https://docs.xilinx.com/r/en-US/ug1399-vitis-hls/Setting-Configuration-Options

[10] Xilinx Inc., Python productivity for Zynq (Pynq) Documentation, 2021 https://pynq.readthedocs.io/en/v2.7.0/getting_started.html

[11] Xilinx Inc., AXI Performance Monitor v5.0 Product Guide (PG037), 2017 https://docs.xilinx.com/v/u/en-US/pg037_axi_perf_mon

[O1] Xilinx Inc., Repozytorium projektu PYNQ, dostęp z dnia 31.10.2022, https://github.com/Xilinx/PYNQ

[O2] Avnet., Repozytorium projektu PYNQ dla układu Ultra96-V2, dostęp z dnia 31.10.2022, https://github.com/Avnet/Ultra96-PYNQ